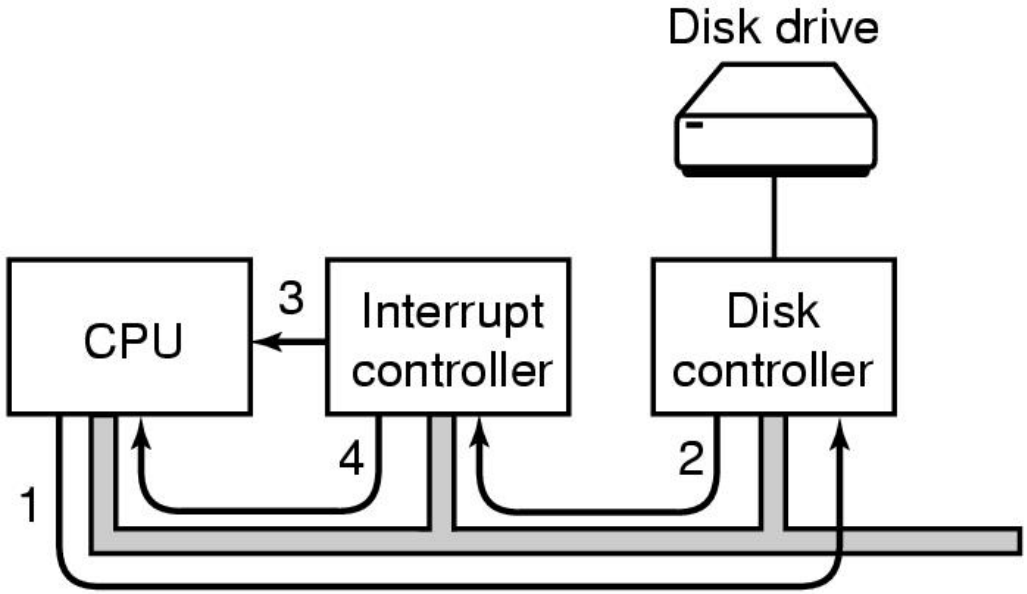
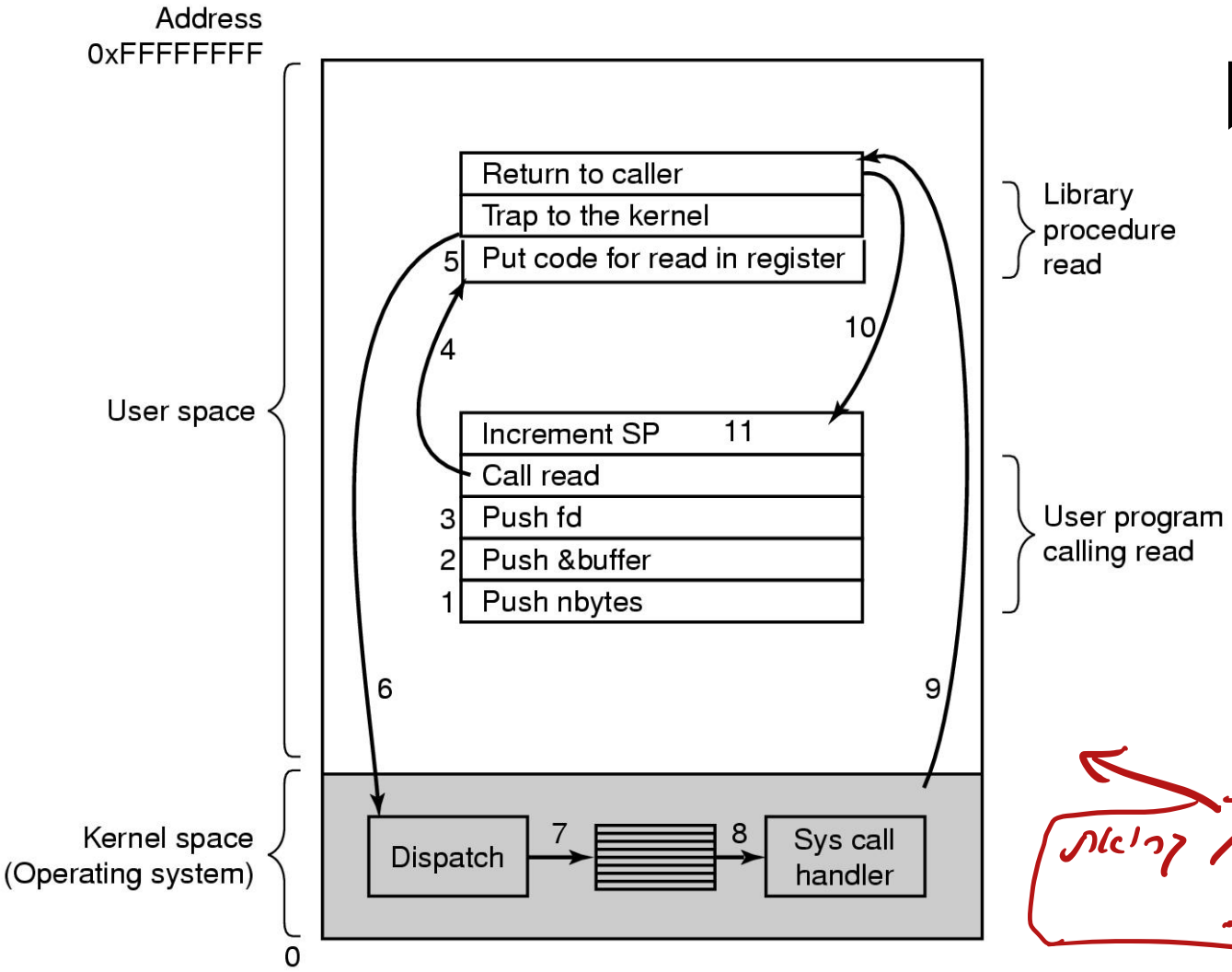
מערכות הפעלה

תהליכים ותהליכונים

פסיקות



קריאות מי



תהליך של קריאת מערכת.

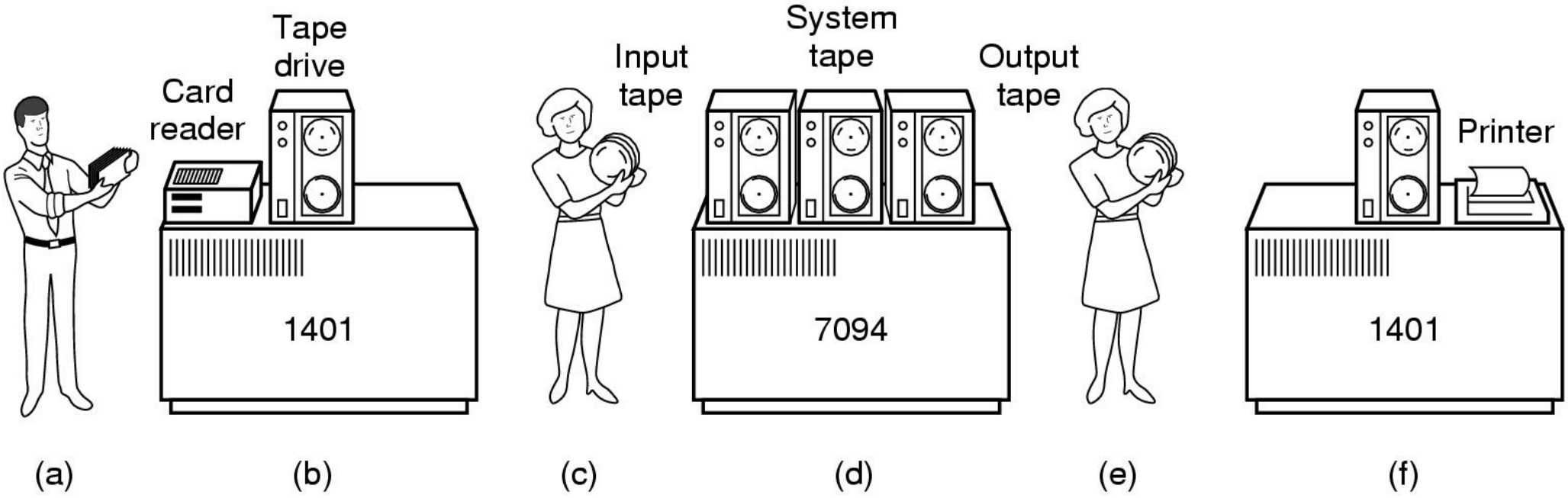
תהליכים

Processes

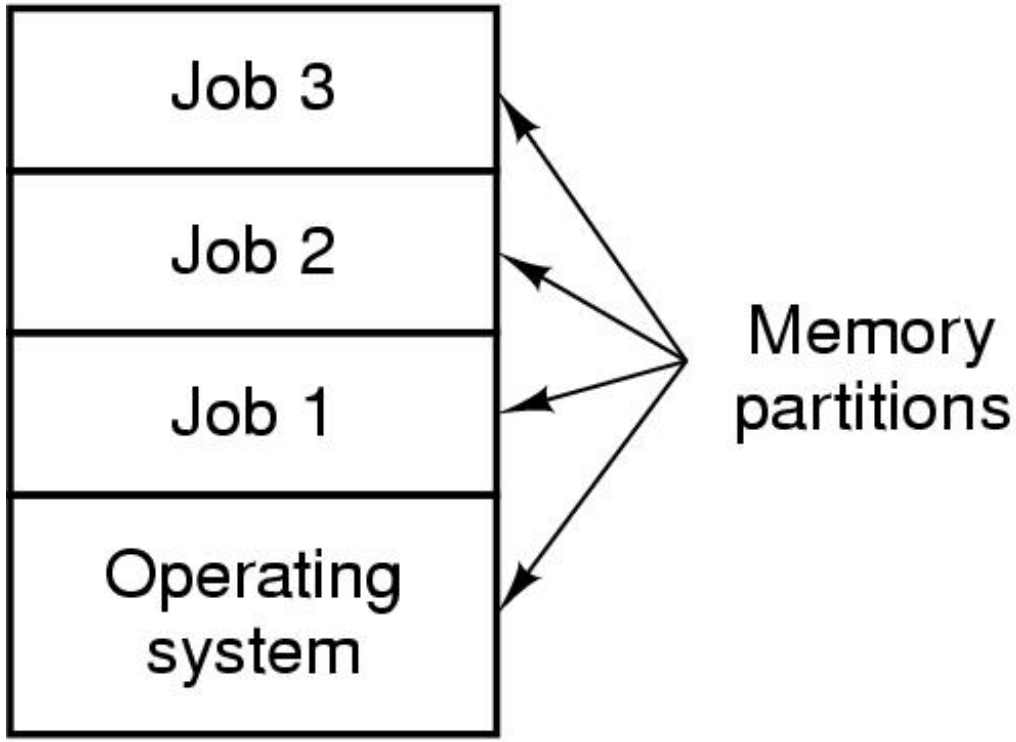
תהליך

מה ההבדל בין תוכנית מחשב לתהליך?

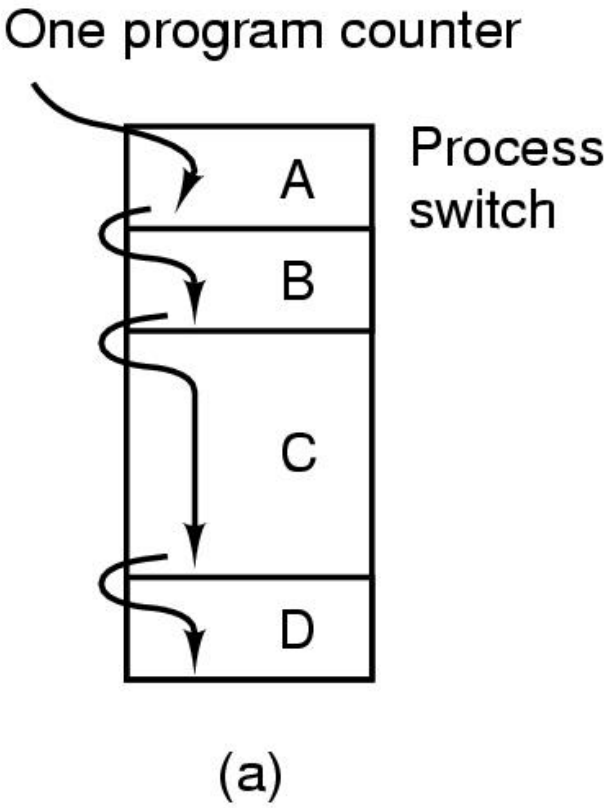
היסטוריה - דור 2



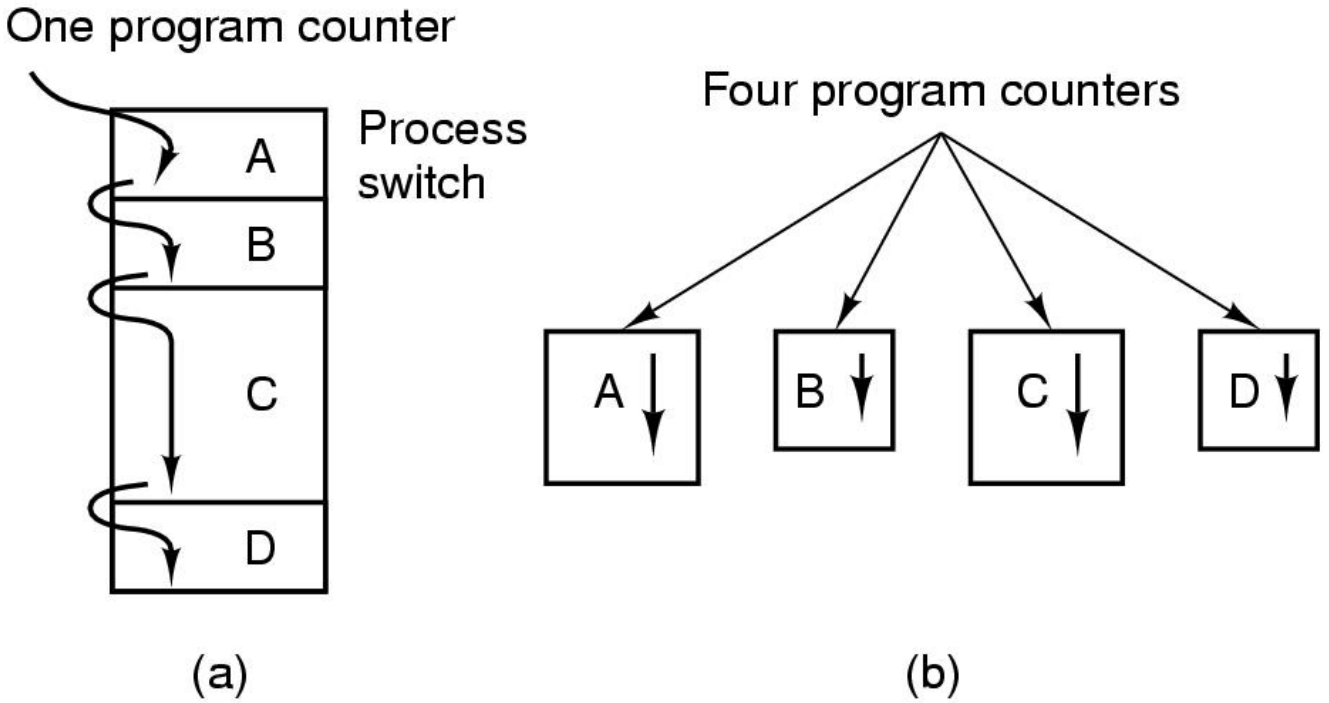
היסטוריה - דור 3



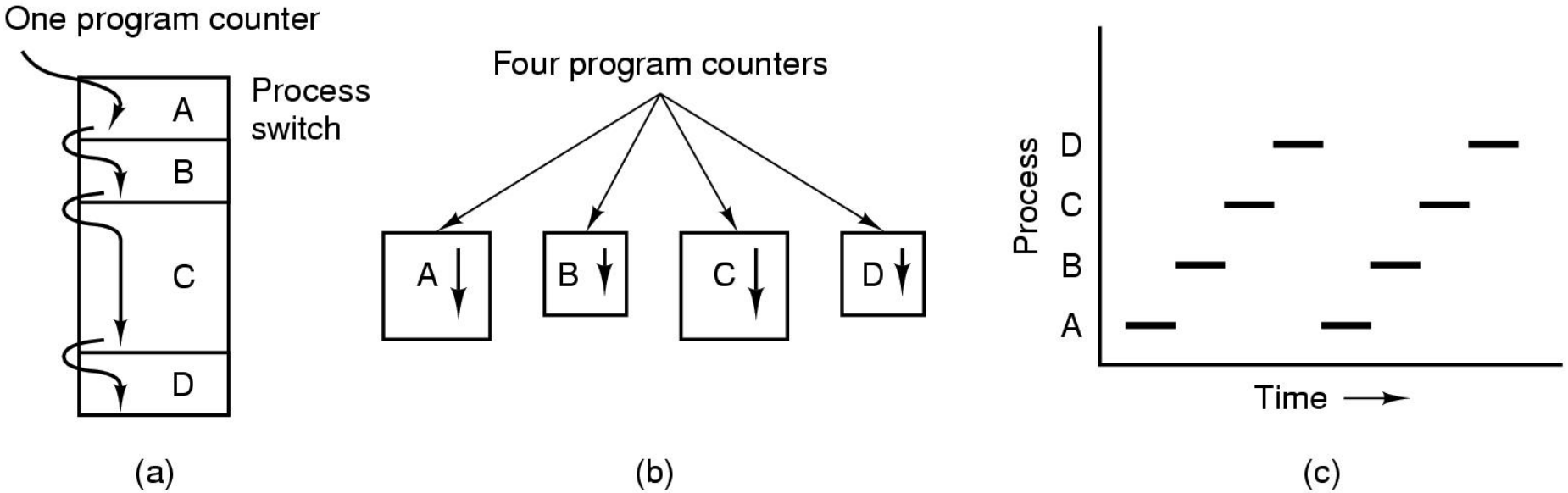
תהליכים



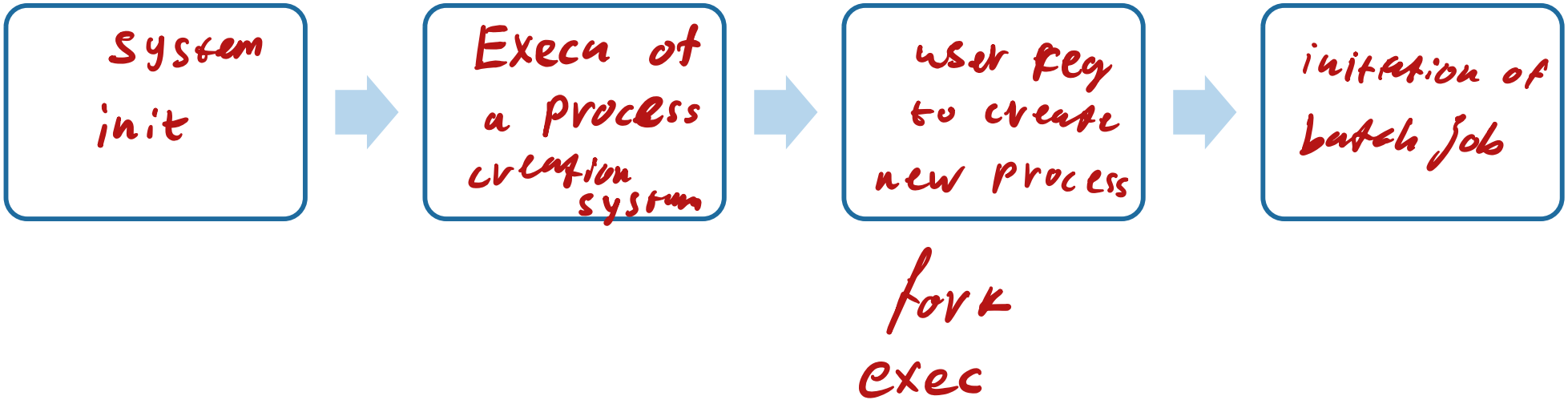
תהליכים


One program counter
A
B
C
D
Process switch
(a)
Four program counters
A
B
C
D
(b)

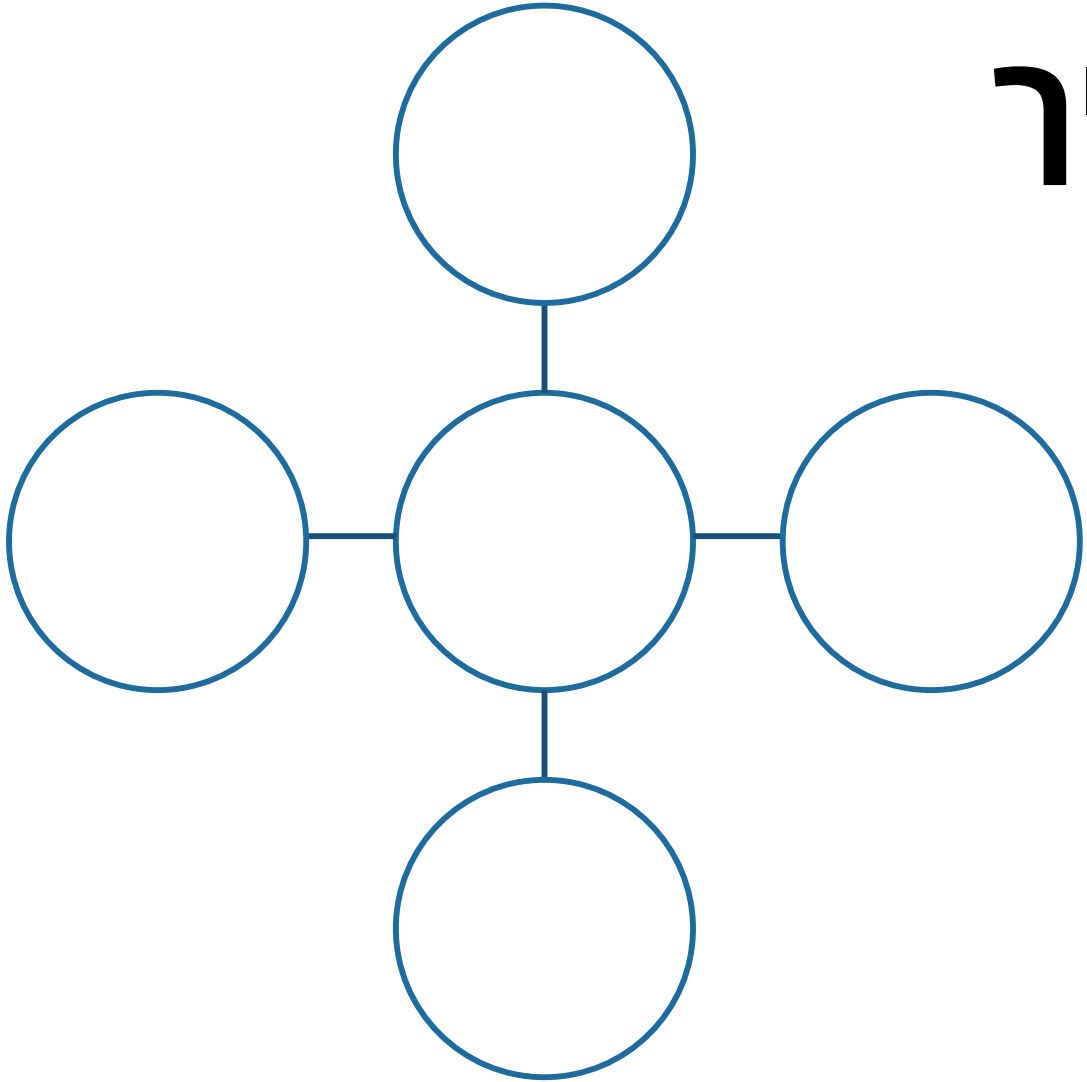
תהליכים


One program counter
Process switch
A
B
C
D
(a)
Four program counters
A
B
C
D
(b)
Process
D
C
B
A
Time
(c)

יצירת תהליך

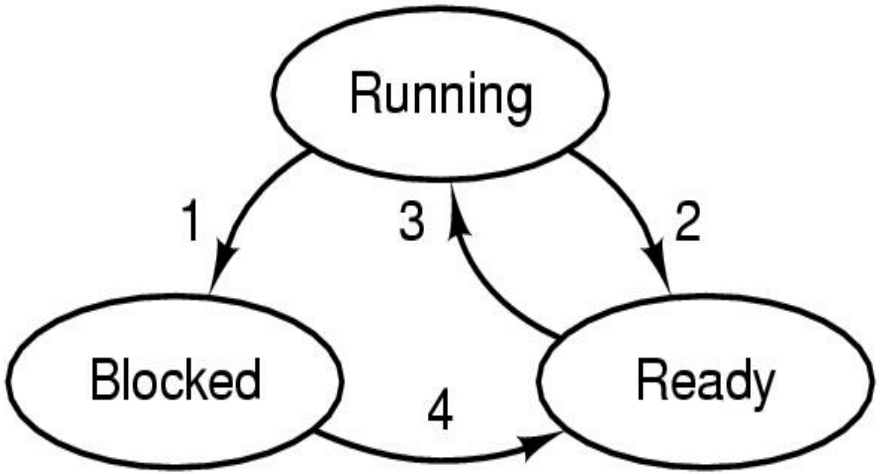

System init
Execu of a process creation system
user req to create new process
initation of batch job
fork
exec

סיום תהליך



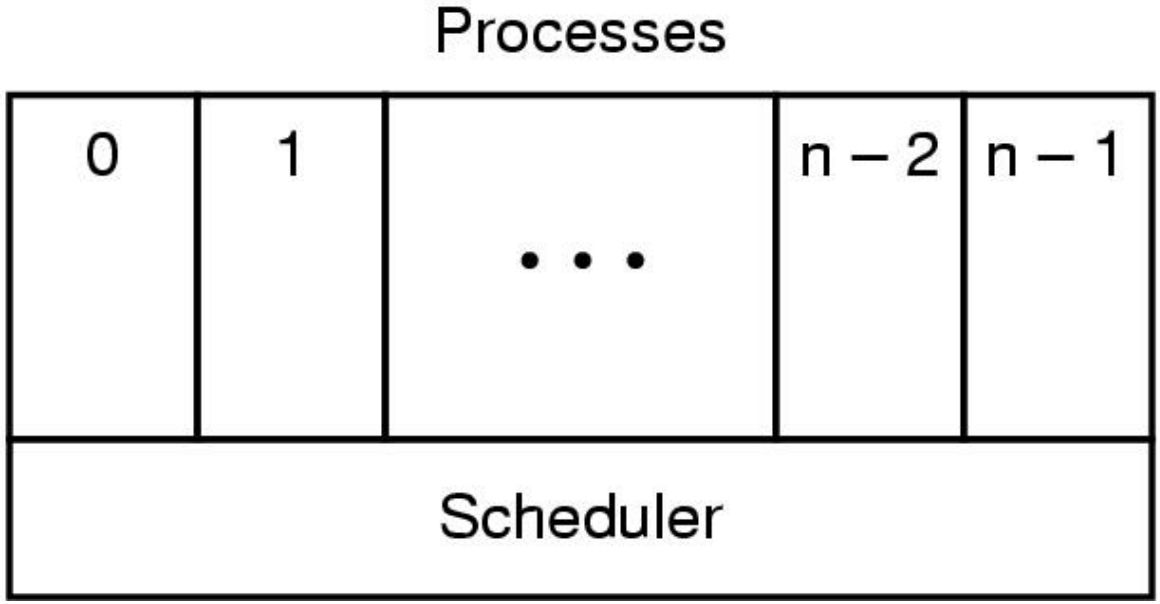
היררכיית תהליכים

מַצָב תהליכים



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

מַצָב תהליכים



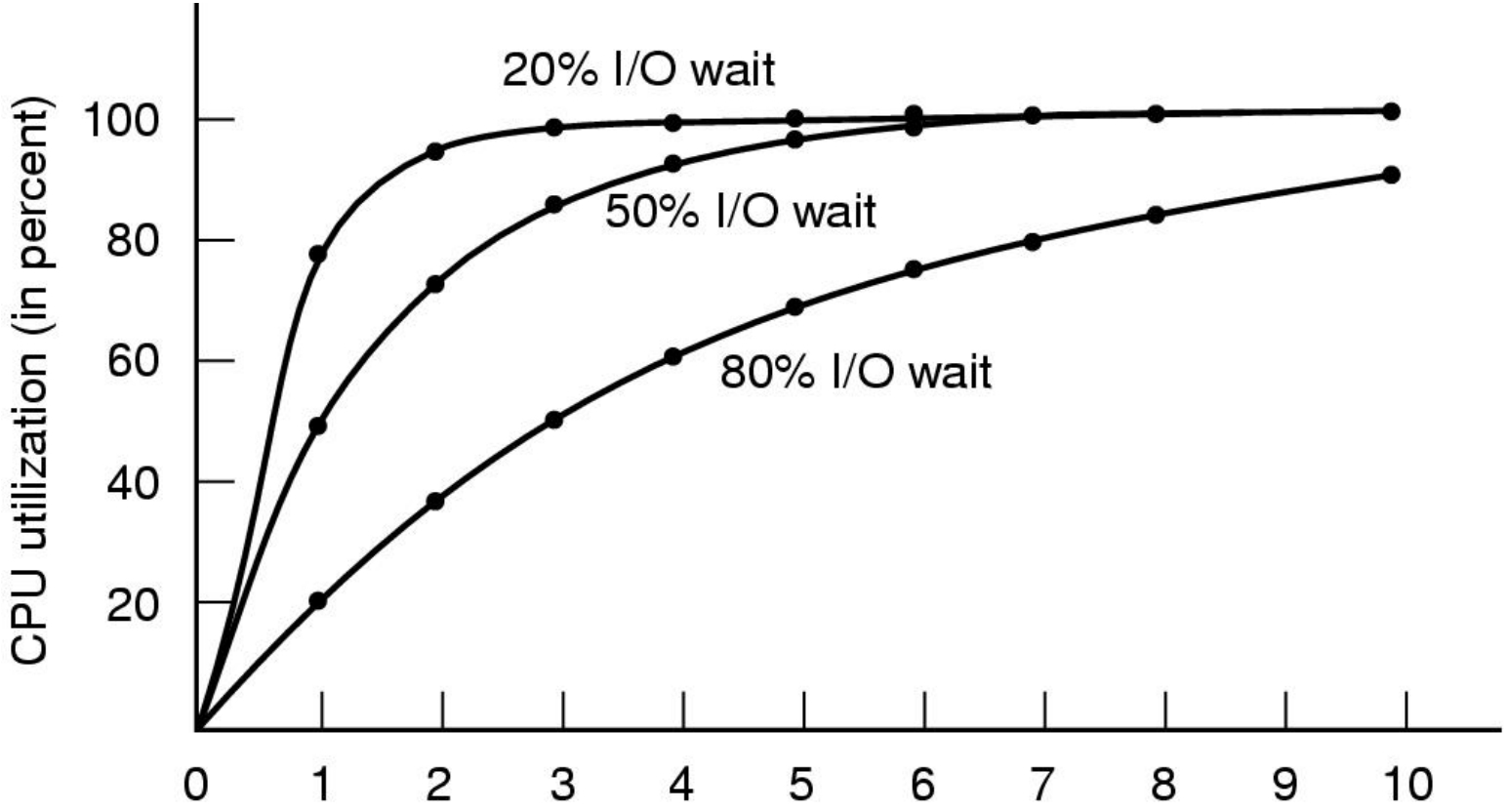
מימוש תהליכים

Process management	Memory management	File management
Registers	Pointer to text segment	Root directory
Program counter	Pointer to data segment	Working directory
Program status word	Pointer to stack segment	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

מימוש תהליכים

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly language procedure saves registers.
4. Assembly language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler decides which process is to run next.
7. C procedure returns to the assembly code.
8. Assembly language procedure starts up new current process.

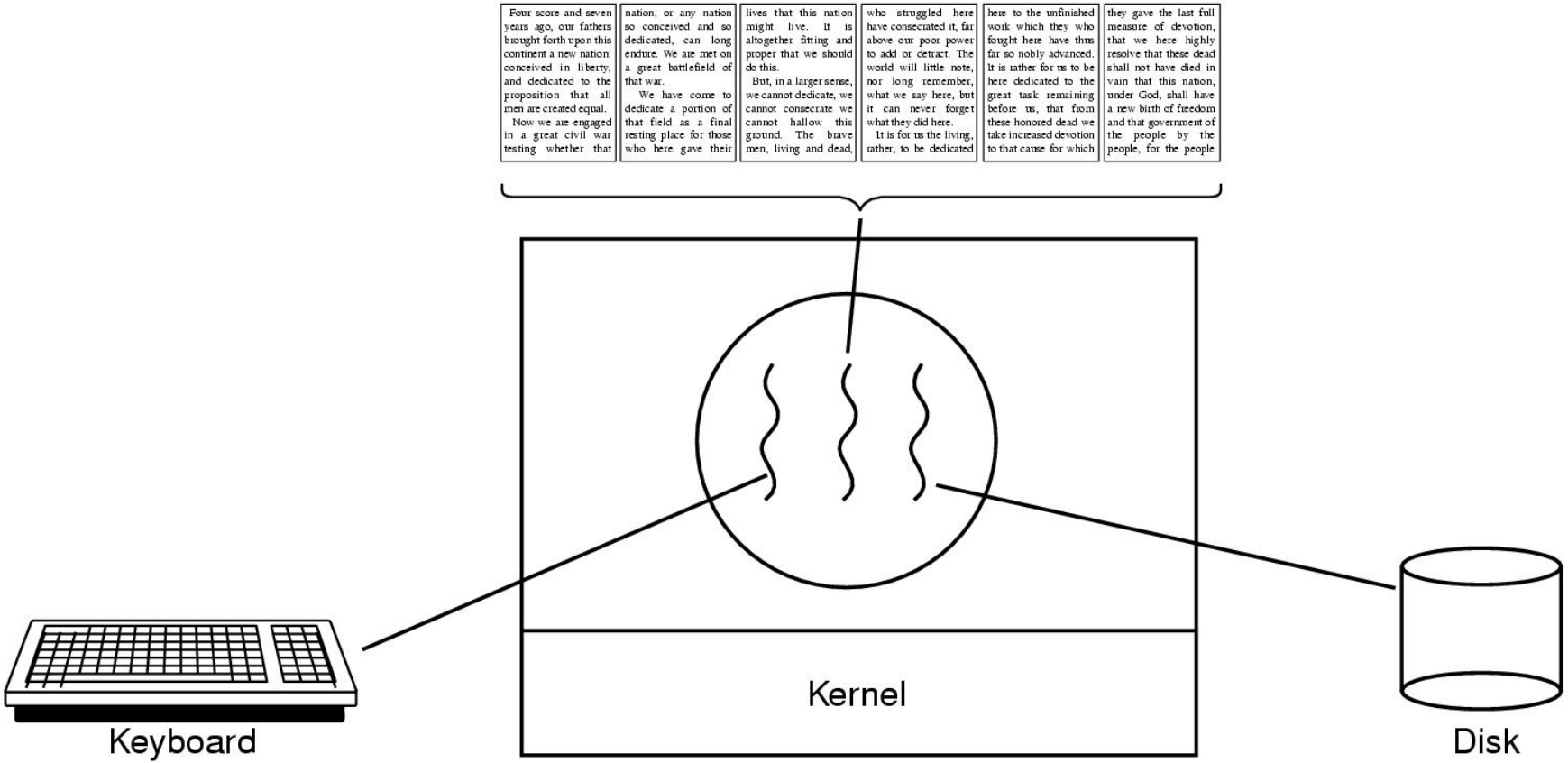
ריבוי תוכניות



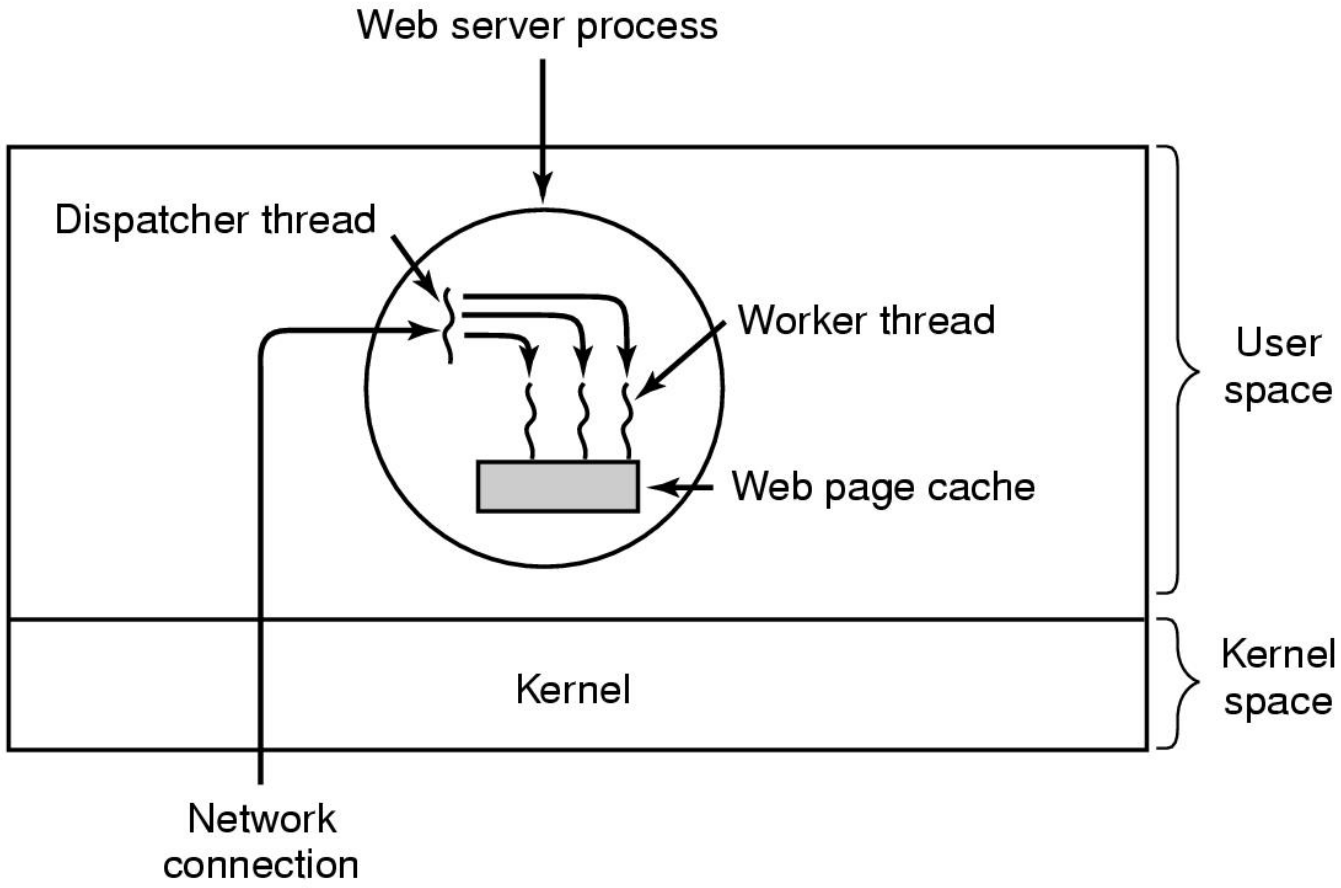
תהליכונים

Threads

תהליכונים


Four score and seven years ago, our fathers brought forth upon this continent a new nation: conceived in liberty, and dedicated to the proposition that all men are created equal.
Now we are engaged in a great civil war testing whether that
nation, or any nation so conceived and so dedicated, can long endure. We are met on a great battlefield of that war.
We have come to dedicate a portion of that field as a final resting place for those who here gave their
lives that this nation might live. It is altogether fitting and proper that we should do this.
But, in a larger sense, we cannot dedicate, we cannot consecrate we cannot hallow this ground. The brave men, living and dead,
who struggled here have consecrated it, far above our poor power to add or detract. The world will little note, nor long remember, what we say here, but it can never forget what they did here.
It is for us the living, rather, to be dedicated
here to the unfinished work which they who fought here have thus far so nobly advanced. It is rather for us to be here dedicated to the great task remaining before us, that from these honored dead we take increased devotion to that cause for which
they gave the last full measure of devotion, that we here highly resolve that these dead shall not have died in vain that this nation, under God, shall have a new birth of freedom and that government of the people by the people, for the people
Kernel
Keyboard
Disk

תהליכונים


Web server process
Dispatcher thread
Worker thread
Web page cache
User space
Kernel
Kernel space
Network connection

תהליכונים

```
while (TRUE) {
  get_next_request(&buf);
  handoff_work(&buf);
}
```

(a)

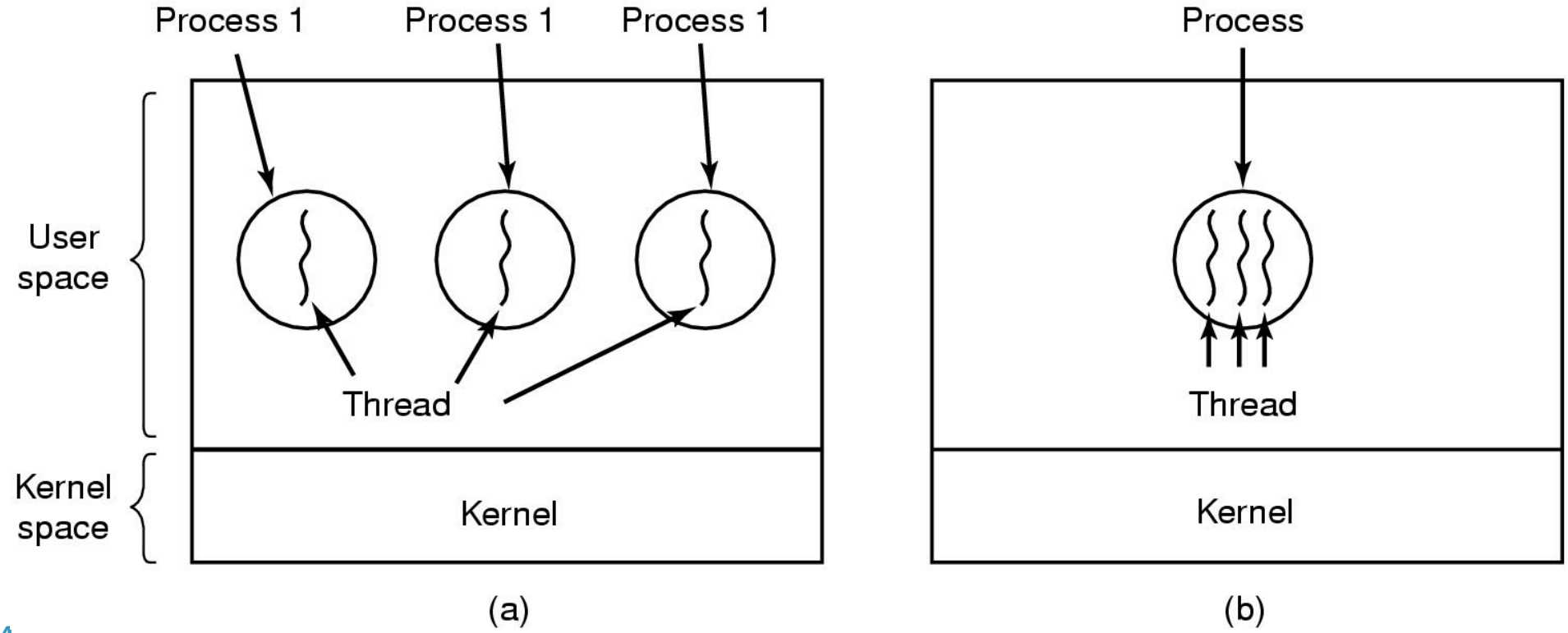
```
while (TRUE) {
  wait_for_work(&buf)
  look_for_page_in_cache(&buf, &page);
  if (page_not_in_cache(&page)
       read_page_from_disk(&buf, &page);
  return_page(&page);
}
```

(b)

תהליכונים

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls, interrupts

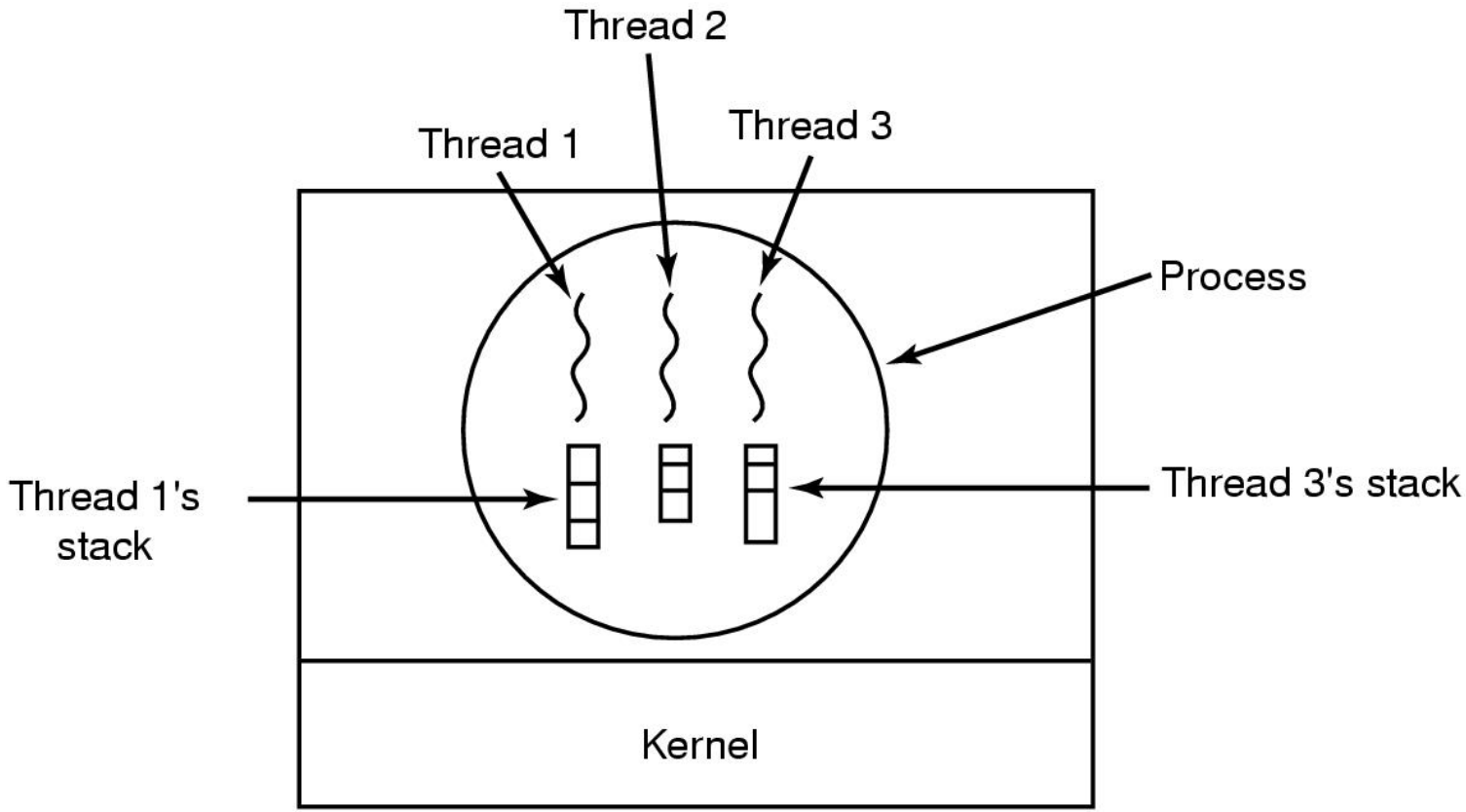
תהליכונים


Process 1
Process 1
Process 1
User space
Thread
Kernel space
Kernel
(a)
Process
Thread
Kernel
(b)

תהליכונים

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

תהליכונים


Thread 2
Thread 1
Thread 3
Process
Thread 1's stack
Thread 3's stack
Kernel

תהליכונים

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run
Pthread_attr_init	Create and initialize a thread's attribute structure
Pthread_attr_destroy	Remove a thread's attribute structure

תהליכון

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS     10

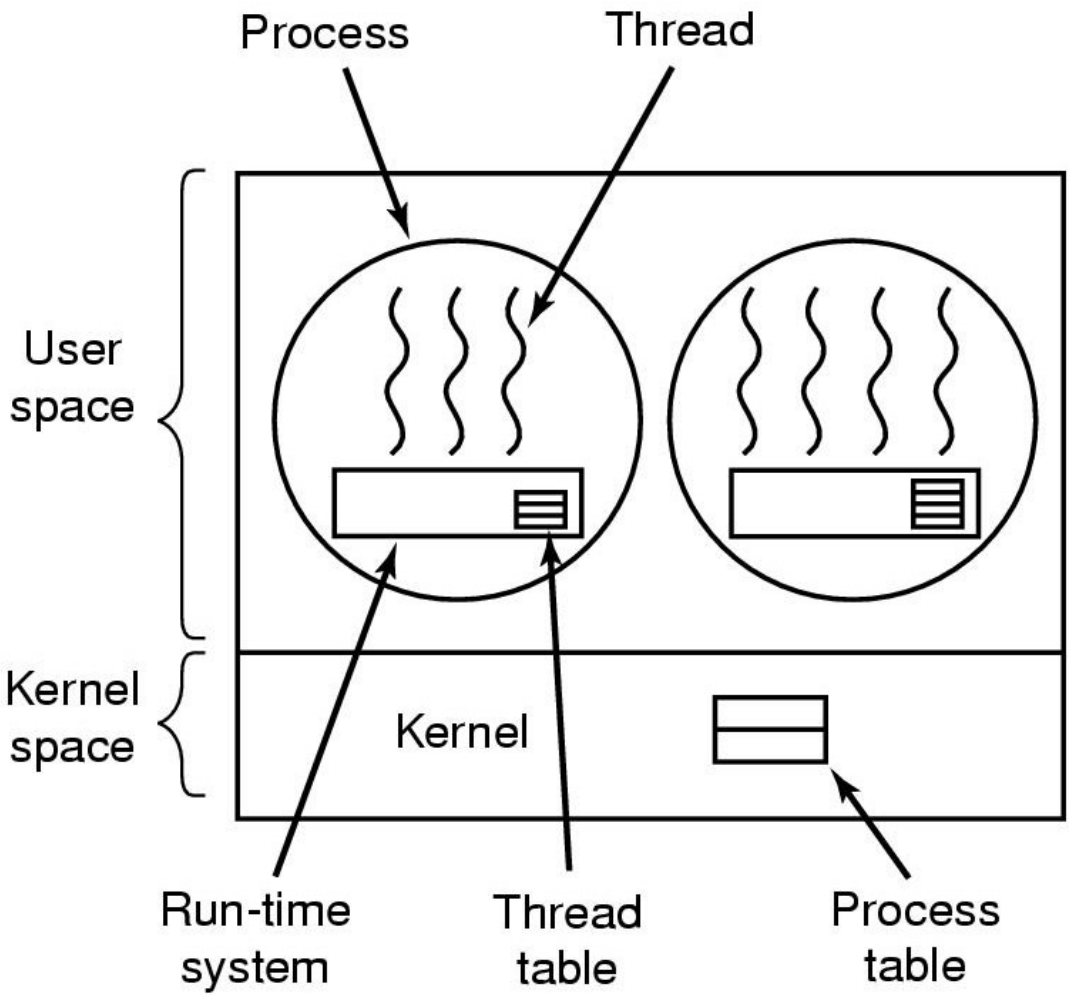
void *print_hello_world(void *tid)
{
        /* This function prints the thread's identifier and then exits. */
        printf("Hello World. Greetings from thread %d0, tid);
        pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
        /* The main program creates 10 threads and then exits. */
        pthread_t threads[NUMBER_OF_THREADS];
        int status, i;

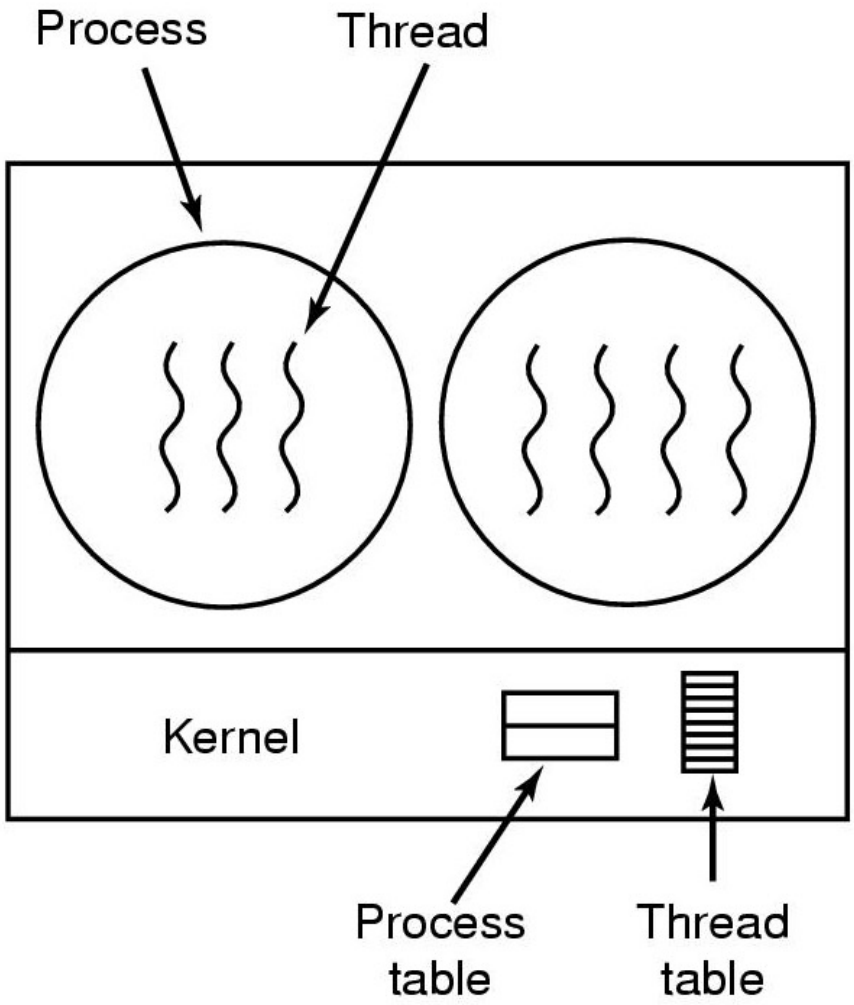
        for(i=0; i < NUMBER_OF_THREADS; i++) {
                printf("Main here. Creating thread %d0, i);
                status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

                if (status != 0) {
                        printf("Oops. pthread_create returned error code %d0, status);
                        exit(-1);
                }
        }
        exit(NULL);
}
```

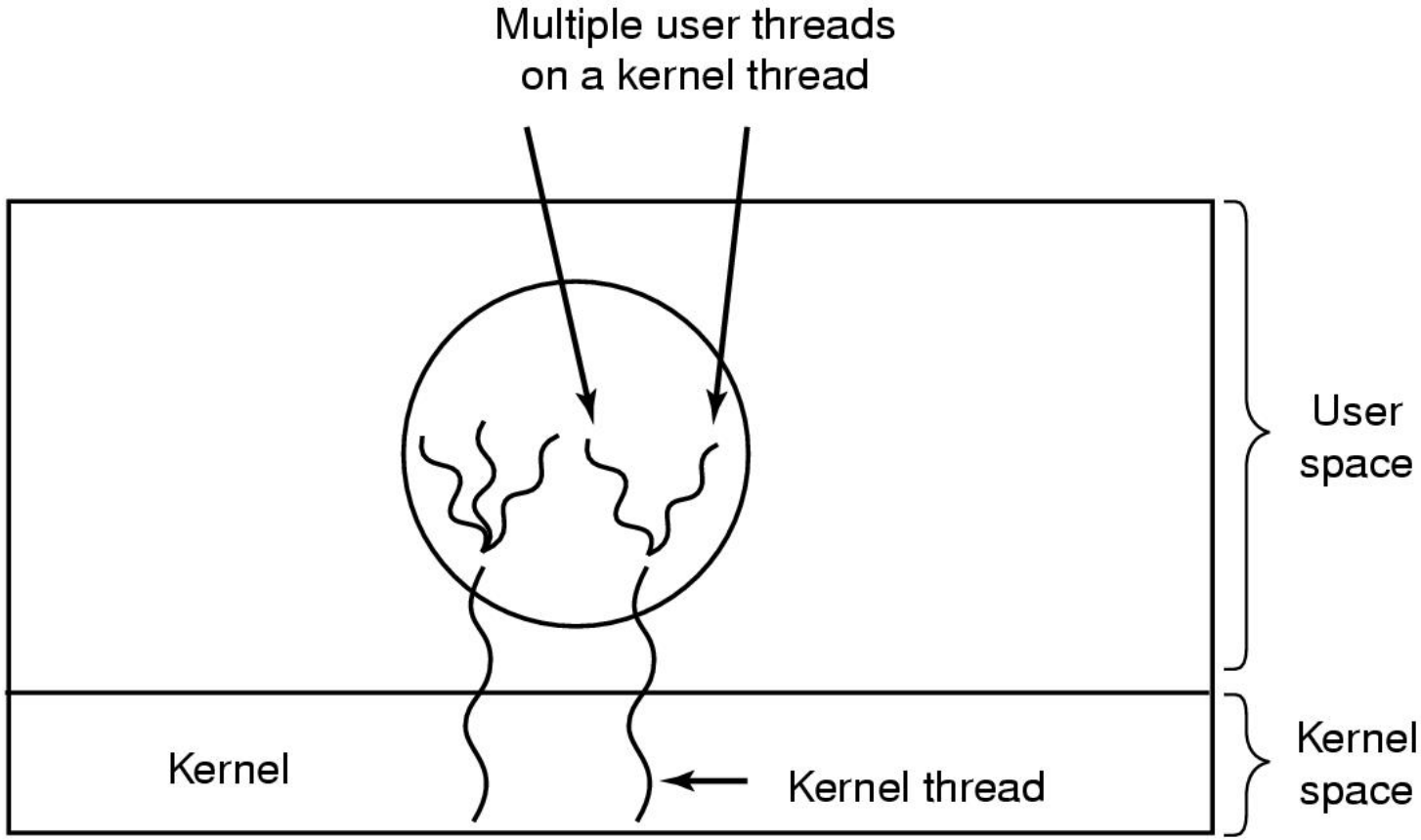
תהליכון


Process
Thread
User space
Kernel space
Kernel
Run-time system
Thread table
Process table

תהליכונים



תהליכונים

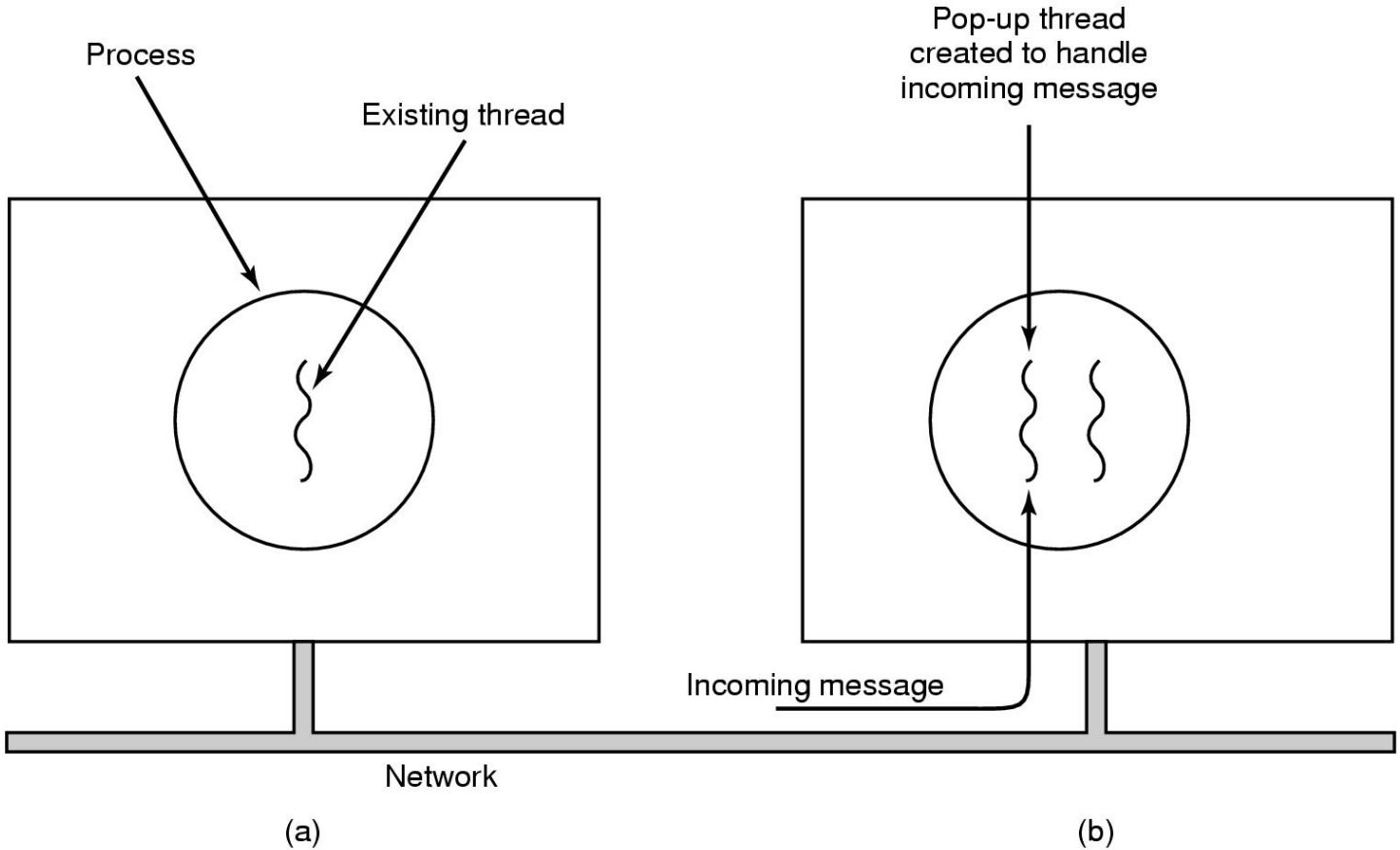

Multiple user threads
on a kernel thread
User
space
Kernel
Kernel thread
Kernel
space

Scheduler Activations

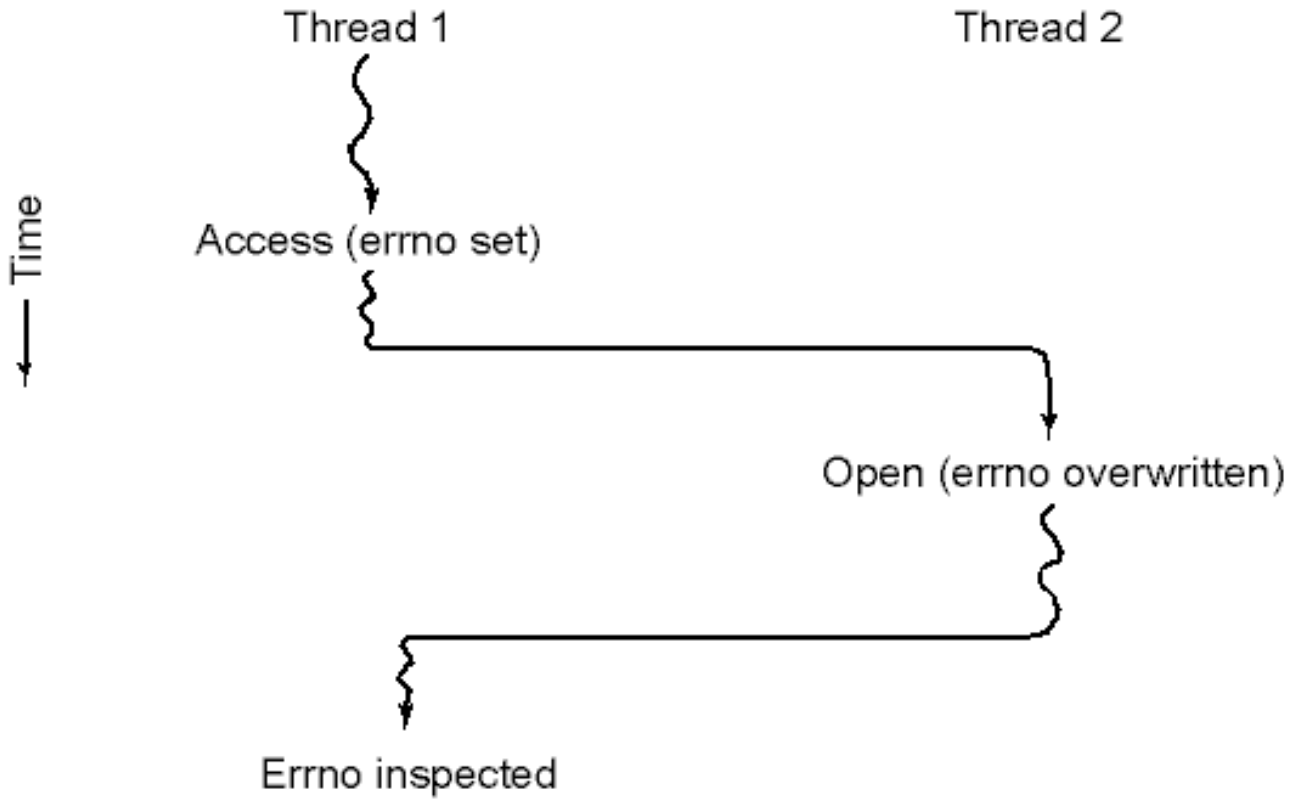
- Goal – mimic functionality of kernel threads
  - gain performance of user space threads
- Avoids unnecessary user/kernel transitions
- Kernel assigns virtual processors to each process
  - lets runtime system allocate threads to processors
- Problem:

  Fundamental reliance on kernel (lower layer) calling procedures in user space (higher layer)

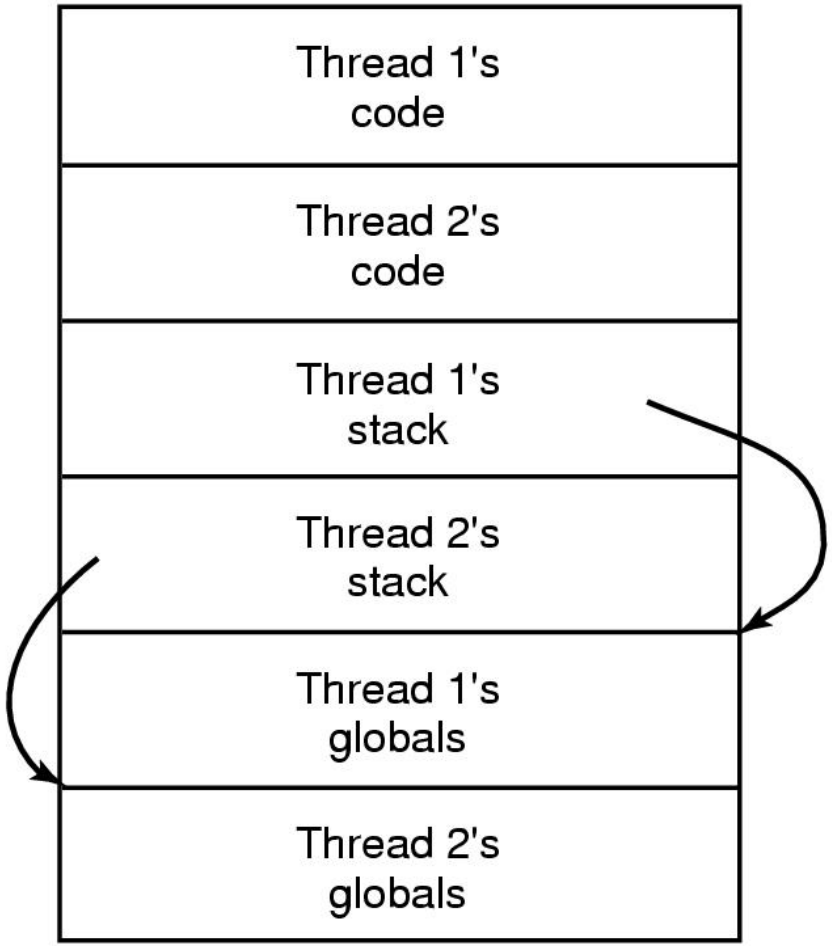
תהליכונים "קופצים"



Making Single-Threaded Code Multithreaded



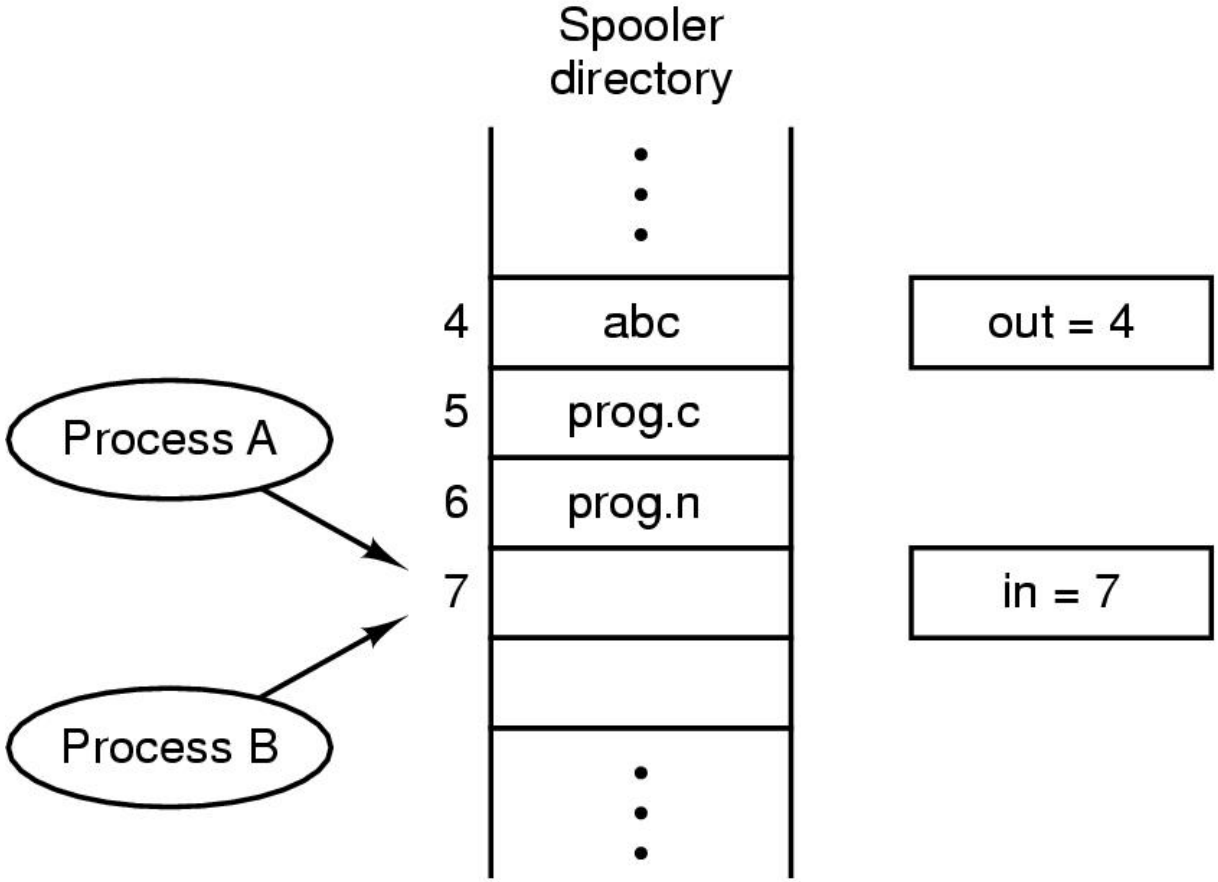
Making Single-Threaded Code Multithreaded



תקשורת בין תהליכים

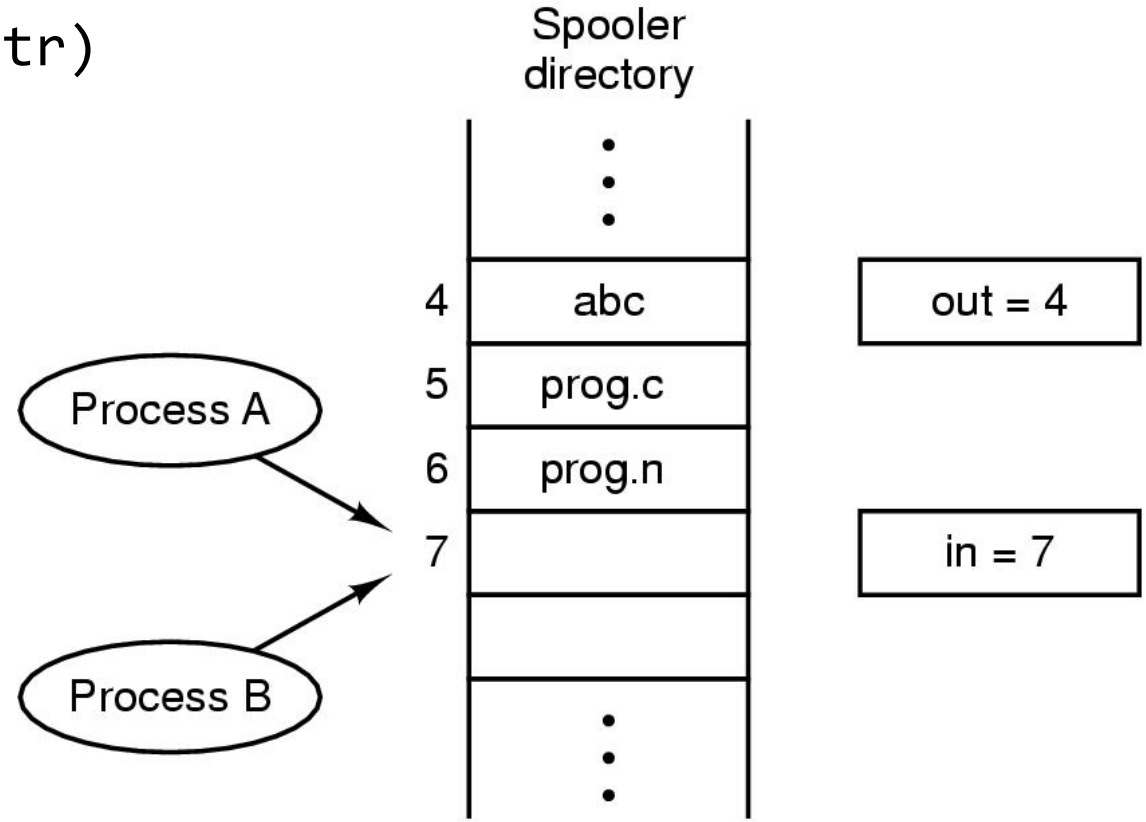
Inter **P**rocess **C**ommunication

תקשורת בין תהליכים



תקשורת בין תהליכים

```
Print(str) 
enque(string str)
{
    a[in]=str
    in++
}
```



תקשורת בין תהליכים

X = יתרה
Y = 800 ש"ח משיכה
Z = יתרה חדשה

(Z=X-Y)

b

```
// i++  == i=i+1;

  00   FF
+ 00   01
  00   00
Cs.w
  01   00
```

a

תקשורת בין תהליכים

X = יתרה
Y = 800 ש"ח משיכה
Z = יתרה חדשה

(Z=X-Y)

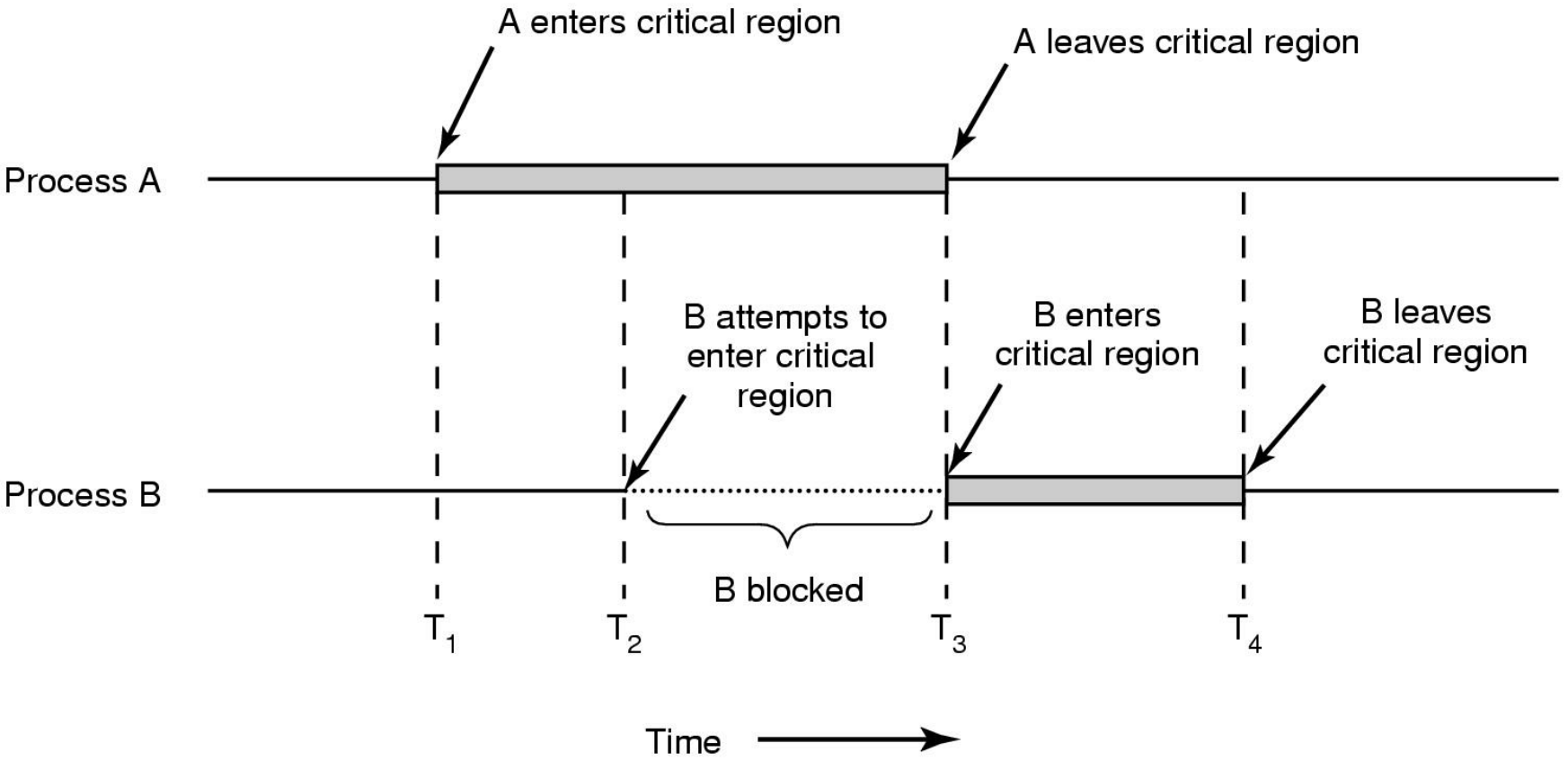
קטעים קריטי


תהליך לא ימתין לנצח כדי להיכנס לקטע קריטי
הפתרון לא יסתמך על השערות הקשורות למהירות או למספר המעבדים
תהליך מחוץ לקטע קריטי לא ימנע מתהליך אחר להיכנס לקטע קריטי
שני תהליכים לא ישהו בו-זמנית בתוך קטע קריטי

קטעים קריטי


הפתרון לא יסתמך
ל השערות
ל למהירות
מעבדים
א ימתין
די להיכנס
קריטי
ץ לקטע
לא ימנע
תהליך אחר
להיכנס לקטע קריטי
שני תהליכים לא
ישהו בו-זמנית
בתוך קטע קריטי

קטעים קריטים



מניעה הדדית הכוללת המתנה פעילה

- מניעת פסיקות
- משתני נעילה
- פתרון התור
- הפתרון של פטרסון
- TSL

מניעת פסיקות

משתני נעילה

תהליך 1	תהליך 2

while(true) {
  // non critical section
  while(lock)
  ;
  lock = 1;
  CriticalSection();
  lock = 0;
  // non critical section
}
	while(true) {
	  // non critical section
	  while(lock)
	  ;
	  lock = 1;
	  CriticalSection();
	  lock = 0;
	  // non critical section
	}

פתרון התור

תהליך 0	תהליך 1

while(true) {
  // non critical section
  while(turn != 0)
  ;
  CriticalSection_0();
  turn = 1;
  // non critical section
}
	while(true) {
	  // non critical section
	  while(turn != 1)
	  ;
	  CriticalSection_1();
	  turn = 0;
	  // non critical section
	}

הפתרון של פטרסון

```
boolean interested[2];
int turn;
interested[0] = interested[1] = false;
turn = 1;
```

תהליך 0	תהליך 1

`while(true) {`
`  // non critical section`
`  interested[0] = true;`
`  turn = 0;`
`  while(interested[1] == true &&`
`  turn == 0)`
`  ;`
`  CriticalSection_0();`
`  interested[0] = false;`
`  // non critical section`
`}`
	`while(true) {`
	`  // non critical section`
	`  interested[1] = true;`
	`  turn = 1;`
	`  while(interested[0] == true &&`
	`  turn == 1)`
	`  ;`
	`  CriticalSection_1();`
	`  interested[1] = false;`
	`  // non critical section`
	`}`

Test and Set Lock (TSL)

```
int TSL(int lock) {
   if(lock == 0) {
      lock = 1;
      return 0;
   }
   return lock;
}
```

Test and Set Lock (TSL)

```
int TSL(int lock) {
   if(lock == 0) {
      lock = 1;
      return 0;
   }
   return lock;
}
```

תהליך 1	תהליך 2

while(true) {
  // non critical section
  while(TSL(lock) != 0)
  ;
  CriticalSection_1();
  lock = 0;
  // non critical section
}
	while(true) {
	  // non critical section
	  while(TSL(lock) != 0)
	  ;
	  CriticalSection_2();
	  lock = 0;
	  // non critical section
	}

הרדמה והערה

- מקיים את ארבעת התנאים של הפתרון הסביר למספר תהליכים אינו גורם להמתנה פעילה
- מסדיר את אופן הטיפול בהליכים שרצו להיכנס לקטע קריטי ולא יכלו לעשות זאת מכיוון שהקטע הקריטי היה תפוס
- הוא אינו מצריך זיהוי מצבים הדורשים היפוך עדיפויות

בעיית היצו

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item( );               /* generate next item */
          if (count == N) sleep( );             /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}


void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep( );             /* if buffer is empty, got to sleep */
          item = remove_item( );                /* take item out of buffer */
          count = count – 1;                    /* decrement count of items in buffer */
          if (count == N – 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

סמפורים

```
down(s) {
   if(s==0)
      place the process on the semaphore queue and move it to the block state;
   if(s>0)
      s--;
}

up(s) {
   if(there is at least one process sleeping in the semaphore queue) {
      wake him up;
      return;
   }
   s++;
}
```

סמפורים

תהליך 0	תהליך 1

while(true) {
   // non critical section
   down(S);
   CriticalSection_0();
   up(S);
   // non critical section
}
	while(true) {
	   // non critical section
	   down(S);
	   CriticalSection_1();
	   up(S);
	   // non critical section
	}

סמפורים

```
#define N 100                                   /* number of slots in the buffer */
typedef int semaphore;                          /* semaphores are a special kind of int */
semaphore mutex = 1;                            /* controls access to critical region */
semaphore empty = N;                            /* counts empty buffer slots */
semaphore full = 0;                             /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {                              /* TRUE is the constant 1 */
        item = produce_item( );                 /* generate something to put in buffer */
        down(&empty);                           /* decrement empty count */
        down(&mutex);                           /* enter critical region */
        insert_item(item);                      /* put new item in buffer */
        up(&mutex);                             /* leave critical region */
        up(&full);                              /* increment count of full slots */
    }
}


void consumer(void)
{
    int item;

    while (TRUE) {                              /* infinite loop */
        down(&full);                            /* decrement full count */
        down(&mutex);                           /* enter critical region */
        item = remove_item( );                  /* take item from buffer */
        up(&mutex);                             /* leave critical region */
        up(&empty);                             /* increment count of empty slots */
        consume_item(item);                     /* do something with the item */
    }
}
```

מנעולים (Mutexes)

```
void MutexLock(mutex) {
   while(TSL(mutex) != 0)   // lock if unlocked
      thread_yield();   // locked, schedule another thread
   return lock;
}

void MutexUnlock(mutex) {
   mutex = 0;
}
```

מבני פיקוח

- מעירה אחד תהליך מתהליכים שיהיה חסום ע"י פעולת signal(c) מבנה פיקוח על התנאי c.
- גורמת להשהיית תהליך כל עוד התנאי c לא יתקיים. פעולת wait(c)

מבני פיקוח

```
monitor ProducerConsumer
        condition full, empty;
        integer count;
        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;
        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count − 1;
                if count = N − 1 then signal(full)
        end;
        count := 0;
end monitor;
```

```
procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;
procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

מבני פיק

```
public class ProducerConsumer {
    static final int N = 100;                   // constant giving the buffer size
    static producer p = new producer( );        // instantiate a new producer thread
    static consumer c = new consumer( );// instantiate a new consumer thread
    static our_monitor mon = new our_monitor( );  // instantiate a new monitor
    public static void main(String args[ ]) {
       p.start( );                              // start the producer thread
       c.start( );                              // start the consumer thread
    }
    static class producer extends Thread {
       public void run( )  {                    // run method contains the thread code
          int item;
          while (true) {                        // producer loop
             item = produce_item( );
             mon.insert(item);
          }
       }
       private int produce_item( ) { ... }      // actually produce
    }
    static class consumer extends Thread {
       public void run( )  {                    run method contains the thread code
          int item;
          while (true) {                        // consumer loop
             item = mon.remove( );
             consume_item (item);
          }
       }
       private void consume_item(int item) { ... }     // actually consume
    }
```

מבני פיקוח

```
static class our_monitor {                  // this is a monitor
    private int buffer[ ] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices
    public synchronized void insert(int val) {
        if (count == N) go_to_sleep( );   // if the buffer is full, go to sleep
        buffer [hi] = val;                // insert an item into the buffer
        hi = (hi + 1) % N;                // slot to place next item in
        count = count + 1;                // one more item in the buffer now
        if (count == 1) notify( );        // if consumer was sleeping, wake it up
    }
    public synchronized int remove( ) {
        int val;
        if (count == 0) go_to_sleep( );   // if the buffer is empty, go to sleep
        val = buffer [lo];                // fetch an item from the buffer
        lo = (lo + 1) % N;                // slot to fetch next item from
        count = count – 1;                // one few items in the buffer
        if (count == N – 1) notify( );    // if producer was sleeping, wake it up
        return val;
    }
    private void go_to_sleep( ) { try{wait( );} catch(InterruptedException exc) {};}
}
}
```

העברת הודעות

- **send**(destination, &message)
  - **receive**(source, &message)

העברת הודעות

```
#define N 100                                   /* number of slots in the buffer */

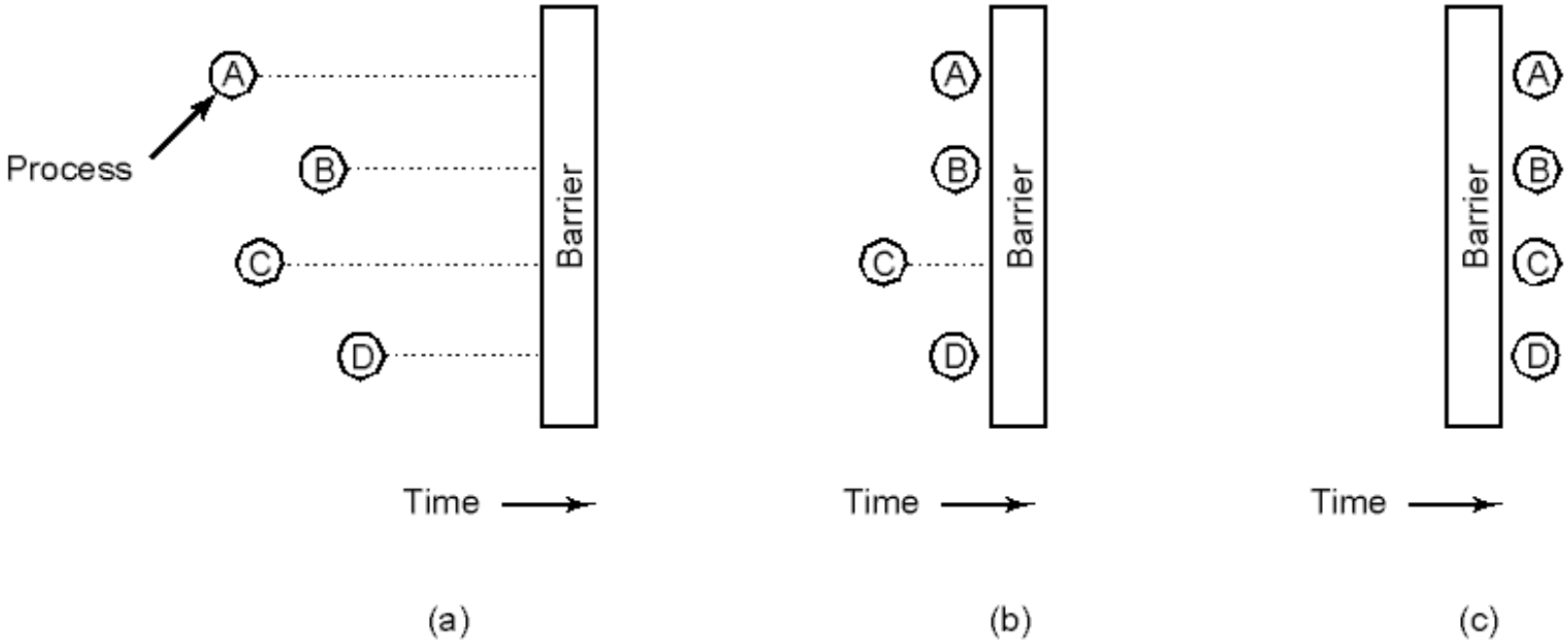
void producer(void)
{
     int item;
     message m;                                 /* message buffer */

     while (TRUE) {
          item = produce_item( );               /* generate something to put in buffer */
          receive(consumer, &m);                /* wait for an empty to arrive */
          build_message(&m, item);              /* construct a message to send */
          send(consumer, &m);                   /* send item to consumer */
     }
}

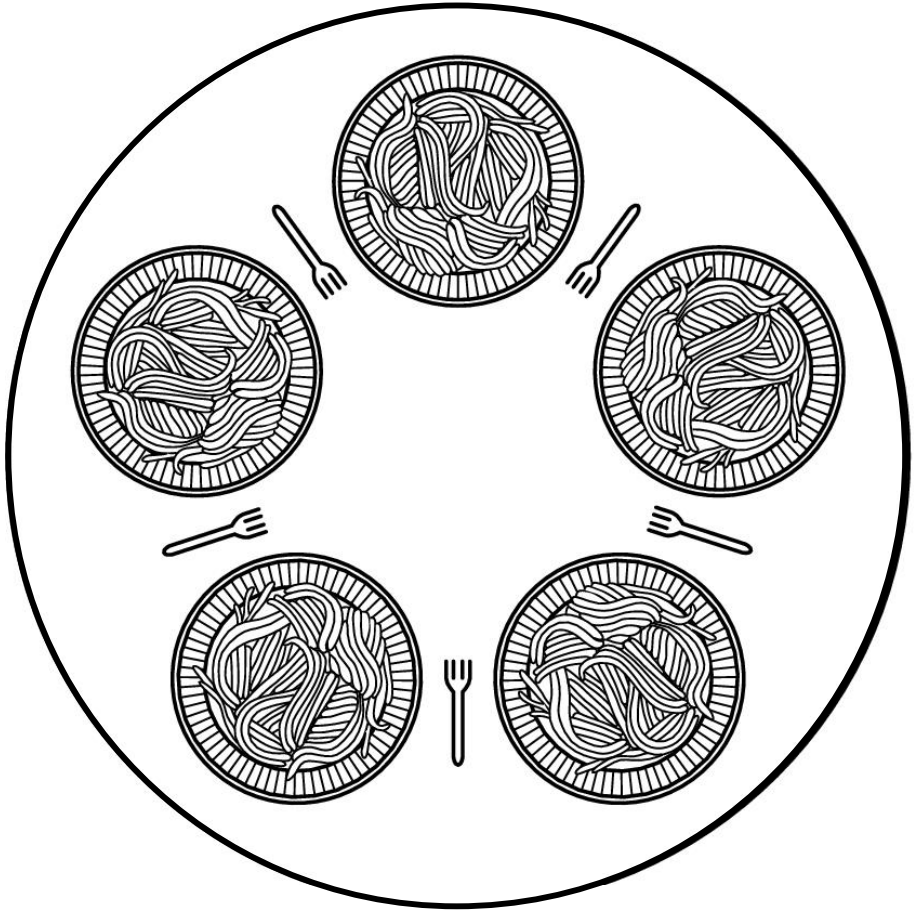
void consumer(void)
{
     int item, i;
     message m;

     for (i = 0; i < N; i++) send(producer, &m);  /* send N empties */
     while (TRUE) {
          receive(producer, &m);                /* get message containing item */
          item = extract_item(&m);              /* extract item from message */
          send(producer, &m);                   /* send back empty reply */
          consume_item(item);                   /* do something with the item */
     }
}
```

מחסומים


Process
A
B
C
D
Barrier
Time
(a)
A
B
C
D
Barrier
Time
(b)
Barrier
A
B
C
D
Time
(c)

בעיית הפילוסופים הסועדים



בעיית הפילוסופים הסועדים

```
#define N 5                         /* number of philosophers */

void philosopher(int i)             /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                   /* philosopher is thinking */
        take_fork(i);               /* take left fork */
        take_fork((i+1) % N);       /* take right fork; % is modulo operator */
        eat( );                     /* yum-yum, spaghetti */
        put_fork(i);                /* put left fork back on the table */
        put_fork((i+1) % N);        /* put right fork back on the table */
    }
}
```

בעיית הפילוסופים הסועדים

```
#define N           5           /* number of philosophers */
#define LEFT        (i+N−1)%N   /* number of i's left neighbor */
#define RIGHT       (i+1)%N     /* number of i's right neighbor */
#define THINKING    0           /* philosopher is thinking */
#define HUNGRY      1           /* philosopher is trying to get forks */
#define EATING      2           /* philosopher is eating */
typedef int semaphore;          /* semaphores are a special kind of int */
int state[N];                   /* array to keep track of everyone's state */
semaphore mutex = 1;            /* mutual exclusion for critical regions */
semaphore s[N];                 /* one semaphore per philosopher */

void philosopher(int i)         /* i: philosopher number, from 0 to N−1 */
{
    while (TRUE) {              /* repeat forever */
        think( );               /* philosopher is thinking */
        take_forks(i);          /* acquire two forks or block */
        eat( );                 /* yum-yum, spaghetti */
        put_forks(i);           /* put both forks back on table */
    }
}
```

בעיית הפילוסופים הסועדים

```
void philosopher(int i)                 /* i: philosopher number, from 0 to N–1 */
{
    while (TRUE) {                      /* repeat forever */
        think( );                       /* philosopher is thinking */
        take_forks(i);                  /* acquire two forks or block */
        eat( );                         /* yum-yum, spaghetti */
        put_forks(i);                   /* put both forks back on table */
    }
}
```

```
void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);                       /* enter critical region */
    state[i] = HUNGRY;                  /* record fact that philosopher i is hungry */
    test(i);                            /* try to acquire 2 forks */
    up(&mutex);                         /* exit critical region */
    down(&s[i]);                        /* block if forks were not acquired */
}

void put_forks(i)                       /* i: philosopher number, from 0 to N–1 */
{
    down(&mutex);                       /* enter critical region */
    state[i] = THINKING;                /* philosopher has finished eating */
    test(LEFT);                         /* see if left neighbor can now eat */
    test(RIGHT);                        /* see if right neighbor can now eat */
    up(&mutex);                         /* exit critical region */
}

void test(i)                            /* i: philosopher number, from 0 to N–1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

בעיית הקוראים והכותבים

```
typedef int semaphore;                  /* use your imagination */
semaphore mutex = 1;                    /* controls access to 'rc' */
semaphore db = 1;                       /* controls access to the database */
int rc = 0;                             /* # of processes reading or wanting to */

void reader(void)
{
    while (TRUE) {                      /* repeat forever */
        down(&mutex);                   /* get exclusive access to 'rc' */
        rc = rc + 1;                    /* one reader more now */
        if (rc == 1) down(&db);         /* if this is the first reader ... */
        up(&mutex);                     /* release exclusive access to 'rc' */
        read_data_base( );              /* access the data */
        down(&mutex);                   /* get exclusive access to 'rc' */
        rc = rc – 1;                    /* one reader fewer now */
        if (rc == 0) up(&db);           /* if this is the last reader ... */
        up(&mutex);                     /* release exclusive access to 'rc' */
        use_data_read( );               /* noncritical region */
    }
}


void writer(void)
{
    while (TRUE) {                      /* repeat forever */
        think_up_data( );               /* noncritical region */
        down(&db);                      /* get exclusive access */
        write_data_base( );             /* update the data */
        up(&db);                        /* release exclusive access */
    }
}
```

בעיית הספר הישן



- במספרה אחת ישנו ספר אחד וכסאות ללקוחות שמחכים
- אם אין לקוחות אז הספר ישן בכיסא
- כשמגיע לקוח חדש והספר ישן הוא מעיר אותו
- כשלקוח מגיע והספר עסוק הוא ישב על אחד הכסאות, אם קיים כיסא פנוי אחרת (הכל מלא) הוא עוזב

בעיית הספר הי

```
#define CHAIRS 5                /* # chairs for waiting customers */

typedef int semaphore;          /* use your imagination */

semaphore customers = 0;        /* # of customers waiting for service */
semaphore barbers = 0;          /* # of barbers waiting for customers */
semaphore mutex = 1;            /* for mutual exclusion */
int waiting = 0;                /* customers are waiting (not being cut) */

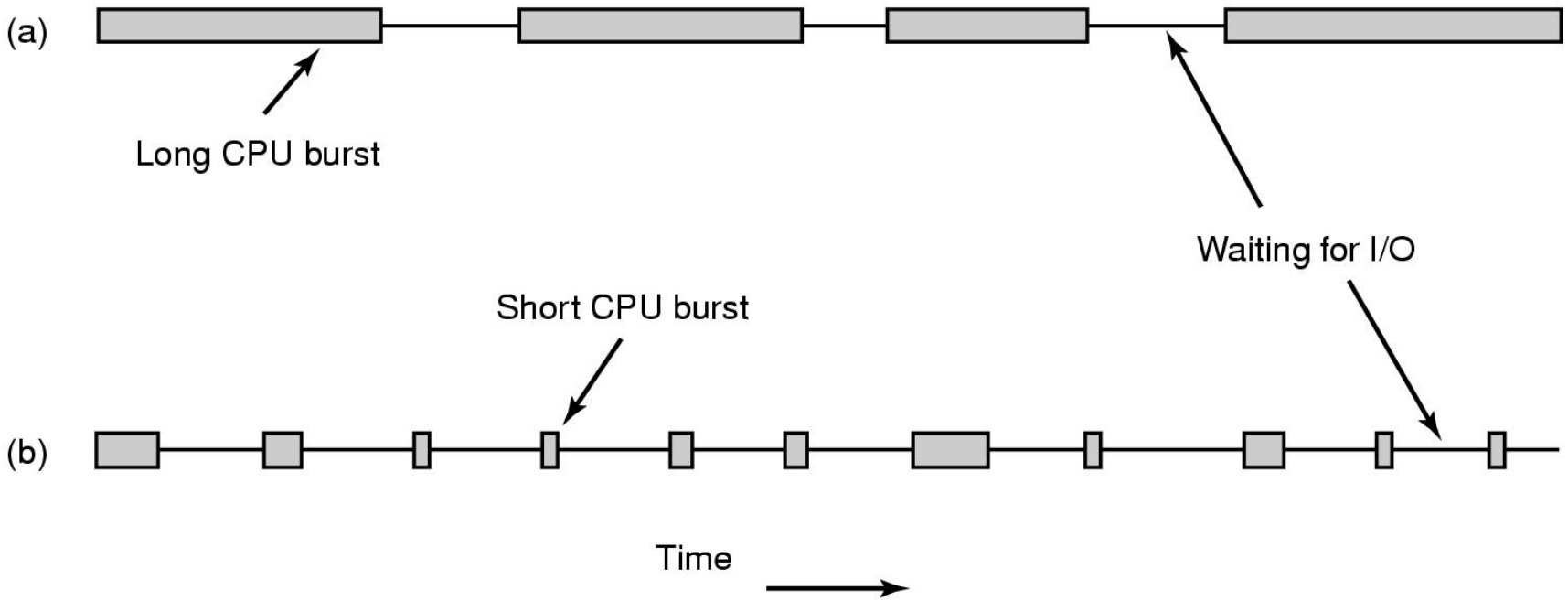
void barber(void)
{
    while (TRUE) {
        down(&customers);       /* go to sleep if # of customers is 0 */
        down(&mutex);           /* acquire access to 'waiting' */
        waiting = waiting – 1;  /* decrement count of waiting customers */
        up(&barbers);           /* one barber is now ready to cut hair */
        up(&mutex);             /* release 'waiting' */
        cut_hair( );            /* cut hair (outside critical region) */
    }
}


void customer(void)
{
    down(&mutex);               /* enter critical region */
    if (waiting < CHAIRS) {     /* if there are no free chairs, leave */
        waiting = waiting + 1;  /* increment count of waiting customers */
        up(&customers);         /* wake up barber if necessary */
        up(&mutex);             /* release access to 'waiting' */
        down(&barbers);         /* go to sleep if # of free barbers is 0 */
        get_haircut( );         /* be seated and be serviced */
    } else {
        up(&mutex);             /* shop is full; do not wait */
    }
}
```

תזמון תהליכים

Scheduling

תזמון תהליכים



תזמון תהליכים

- Batch
  - First-come First-Served
  - Shortest hob first
  - Shortest remaining Time Next
- Interactive
  - Round-Robin scheduling
  - Priority scheduling & Multiple queues
  - Shortest process next
  - Aging
  - Guaranteed scheduling
  - Fair-share scheduling
  - Lottery scheduling
- Real time

קריטריונים להערכה של אלגוריתמי תזמון

1. הגינות (Fairness)
2. אכיפת מדיניות (Policy enforcement)
3. **איזון** (Balance)

קריטריונים להערכה של אלגוריתמי תזמון

All systems

Fairness - giving each process a fair share of the CPU
Policy enforcement - seeing that stated policy is carried out
Balance - keeping all parts of the system busy

Batch systems

Throughput - maximize jobs per hour
Turnaround time - minimize time between submission and termination
CPU utilization - keep the CPU busy all the time

Interactive systems

Response time - respond to requests quickly
Proportionality - meet users' expectations

Real-time systems

Meeting deadlines - avoid losing data
Predictability - avoid quality degradation in multimedia systems

תזמון במערכות אֲצְוָוה

- First-Come First-Served
- Shortest Job First
- Shortest Remaining Time Next

תזמון במערכות אַצְוָה

זמן שהייה ממוצע	זמן שהייה	סיום	התחלה	CPU	זמן הגעה	תהליך	תזמון

זמן שהייה ממוצע	זמן שהייה	סיום	התחלה	CPU	זמן הגעה	תהליך	תזמון

תזמון במערכות אַצְוָה

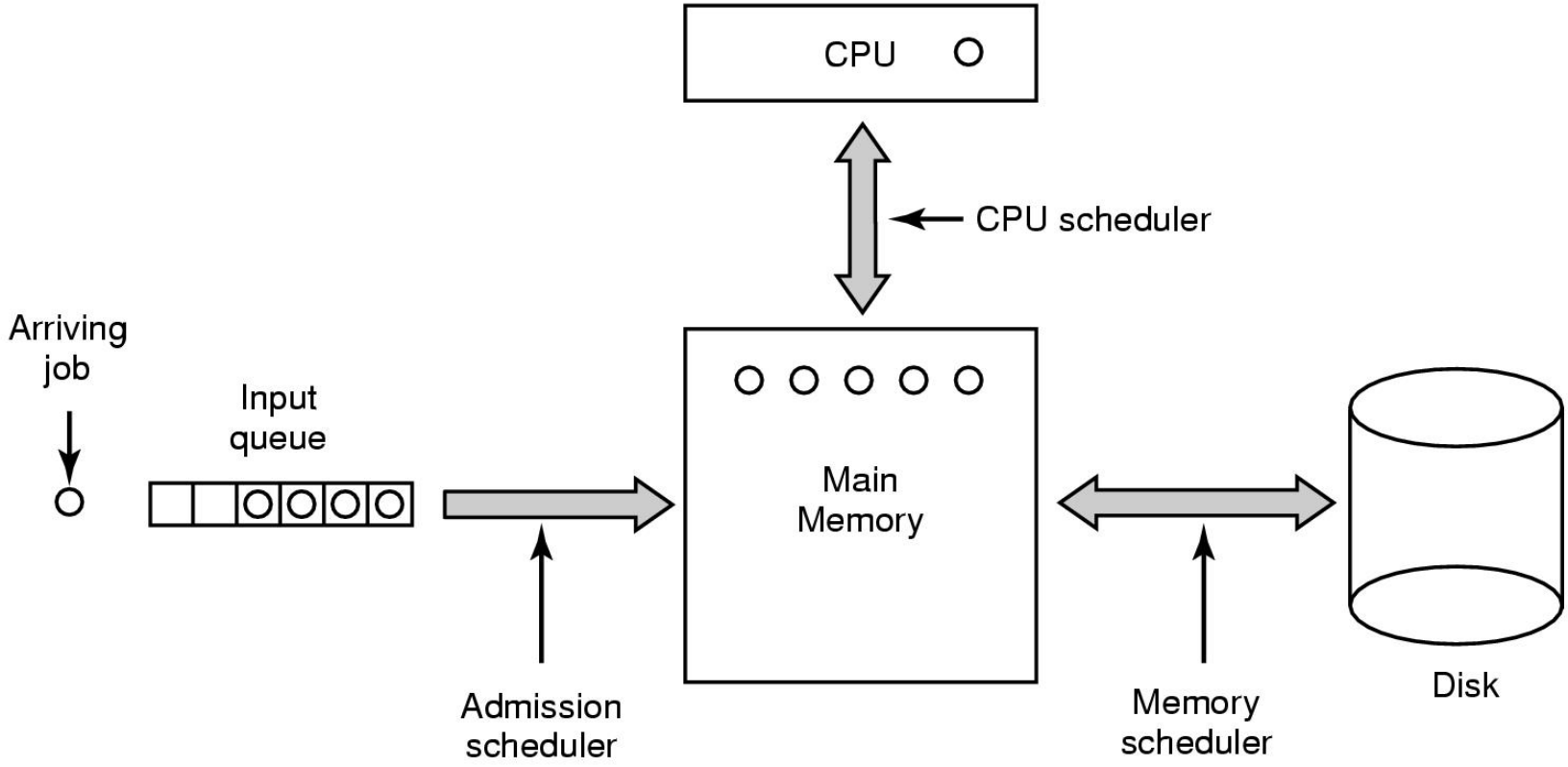
זמן שהייה ממוצע	זמן שהייה	סיום	התחלה	CPU	זמן הגעה	תהליך	תזמון

זמן שהייה ממוצע	זמן שהייה	סיום	התחלה	CPU	זמן הגעה	תהליך	תזמון

Shortest Remaining Time Next

זמן שהייה ממוצע	זמן שהייה	סיום	התחלה	CPU	זמן הגעה	תהליך	תזמון

תזמון במערכות אַצְוָוה

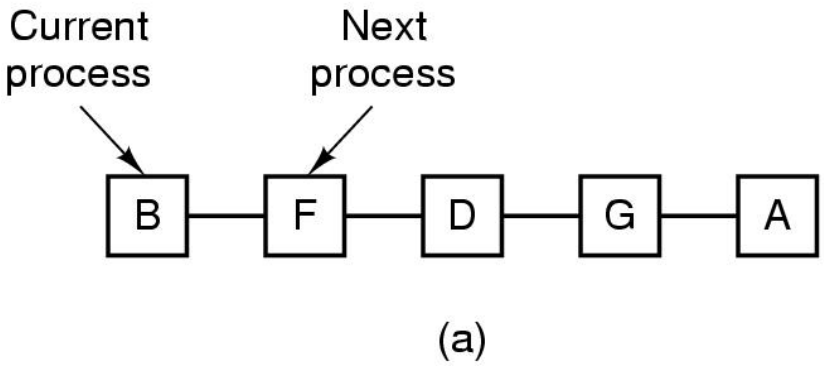


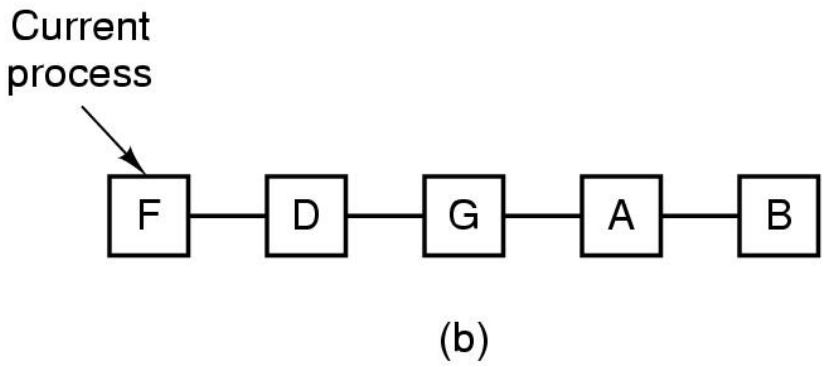
תזמון במערכות אינטראקטיביות

- Round-Robin scheduling
- Priority scheduling & Multiple queues
- Shortest process next
- Aging
- Guaranteed scheduling
- Fair-share scheduling
- Lottery scheduling

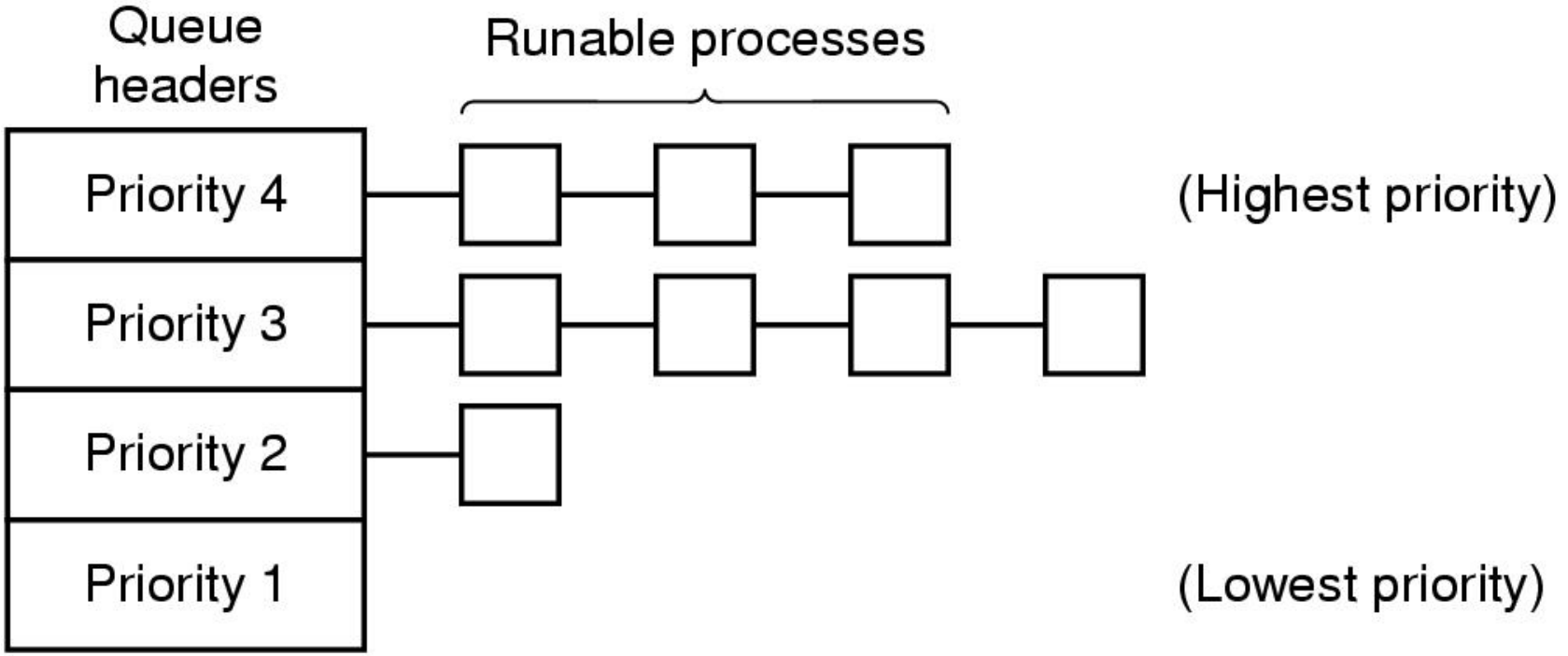
תזמון במערכות אינטראקטיביות

Round Robin





תזמון במערכות אינטראקטיביות



תזמון במערכות אינטראקטיביות

Shortest process next

Aging

$$E(i+1) = (1-a)\,T(i) + a\,E(i)$$

תזמון במערכות אינטראקטיביות

Guaranteed scheduling

תזמון במערכות אינטראקטיביות

Lottery

תזמון במערכות אינטראקטיביות

Fair share scheduling

תזמון במערכות זמן אמת

Schedulable real-time system

- Given
  - m periodic events
  - event i occurs within period Pi and requires Ci seconds
- Then the load can only be handled if

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

תרגול 1

מה תהיה התחזית לריצה הבאה אם האלגוריתם לתזמון תהליכים הוא גירסה של SJF (Shortest Job First) המשתמש בטכניקת ה- aging כאשר:

- הריצות שהיו בפועל (משמאל לימין) הן 80, 40, 30
- a = ½
- צפי זמן התחלתי הוא 80

א. 50
ב. 25
ג. 30

תרגול 2

גורמת ל quantum הגדלת אורך יחידת ה
בשיטת התזמון round robin:

א. הגדלת ניצולת ה CPU
ב. שיפור המקביליות המדומה
ג. הקטנת זמן התגובה לתהליכים אינטראקטיביים
ד. הגדלת התקורה של (overhead) context

תרגול 3

כיצד אפשר לתזמן תהליכים בעזרת lottery scheduling כך שיתקבל בקירוב תזמון round robin?

א. ע"י מתן כרטיסים שמספרם מהווה סדרה הנדסית (geometric progression)
ב. ע"י מתן כרטיסים שמספרם מהווה סדרה חשבונית (arithmetic progression)
ג. ע"י חלוקת כרטיסים במידה שווה בין כל התהליכים

תרגול 4

מה תהיה התחזית לריצה הבאה אם האלגוריתם לתזמון תהליכים הוא גירסה של SJF (Shortest Job First) המשתמש בטכניקת ה- aging כאשר:

- הריצות שהיו בפועל (משמאל לימין) הן 4 ,36 4, 12
- a = ¼
- צפי זמן התחלתי הוא 36

א. 10.5

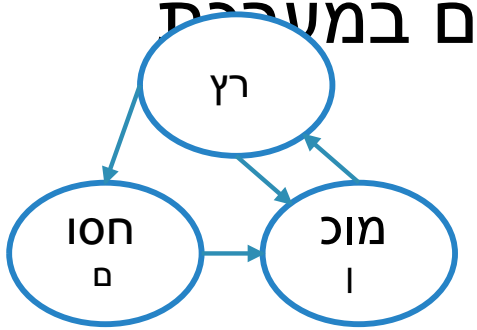
ב. 12

ג. 13.5

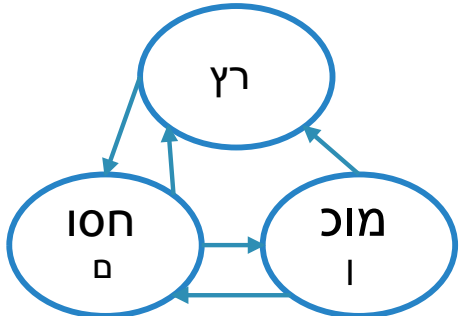
תרגול 5

לפניכם מספר דיאגרמות המתארות מעבר בין שלושת המצבים שבהם יכול להיות כל תהליך: רץ (Running), מוכן (Ready) וחסום (Blocked).
מהי הדיאגרמה המתאימה לגבי תהליכים במערכת UNIX?

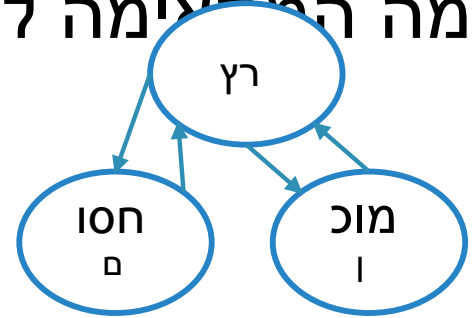
1



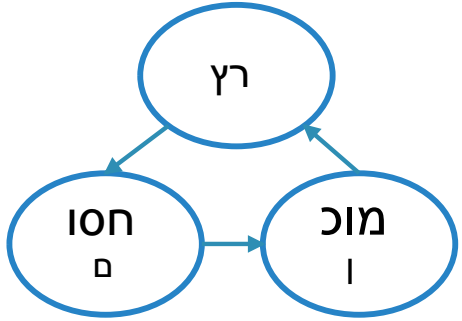
2



3



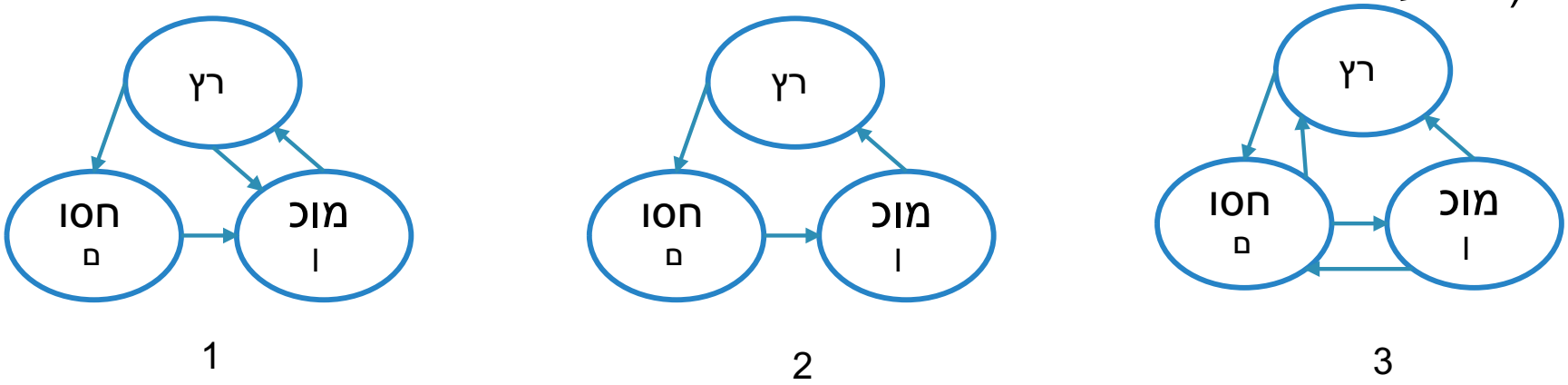
4



תרגול 6

לפניכם מספר דיאגרמות המתארות מעבר בין שלושת המצבים שבהם יכול להיות כל תהליך: רץ (Running), מוכן (Ready) וחסום (Blocked).

מהי הדיאגרמה המתאימה לגבי תהליכים במערכת שתומכת בתהליכי אצווה (batch processing)? (במערכת כזאת אין החלפת תהליכים אלה אם כן תהליך בעצמו מבצע פעולה שמעבירה אותו למצב חסום).



תרגול 7

במערכת זמן קיימות משימות לפי הטבלה הנתונה.
ניתן לתזמן לכל היותר כמה משימות מסוג F במערכת (בנוסף לאלה שכבר קיימות) כך שהמערכת תוכל עדיין לתזמן את כל המשימות? ניתן להניח כי זמן ההחלפת התהליכים הוא זניח

Task	Execution time	Period
A	1	4
B	1	4
C	1	8
D	1	8
E	1	16
F	1	16

א. 1
ב. 2
ג. 3

סיכום

סיכום



תהליכים ותהליכונים

- פסיקות
- System calls



תקשורת בין תהליכים

- מניעה הדדית
- המתנה פעילה
- הרדמה והערה
- סמפורים



תזמון תהליכים

- מערכות אצווה
- מערכות אינטראקטיביות
- מערכות זמן אמת



תודה